arXiv:2402.03300v3 [cs.CL] 27 Apr 2024

DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models

Zhihong Shao[1,2*†], Peiyi Wang[1,3*†], Qihao Zhu[1,3*†], Runxin Xu[1], Junxiao Song[1]
Xiao Bi[1], Haowei Zhang[1], Mingchuan Zhang[1], Y.K. Li[1], Y. Wu[1], Daya Guo[1*]

[1]DeepSeek-AI, [2]Tsinghua University, [3]Peking University

{zhihongshao,wangpeiyi,zhuqh,guoday}@deepseek.com
https://github.com/deepseek-ai/DeepSeek-Math

Abstract

Mathematical reasoning poses a significant challenge for language models due to its complex and structured nature. In this paper, we introduce DeepSeekMath 7B, which continues pre-training DeepSeek-Coder-Base-v1.5 7B with 120B math-related tokens sourced from Common Crawl, together with natural language and code data. DeepSeekMath 7B has achieved an impressive score of 51.7% on the competition-level MATH benchmark without relying on external toolkits and voting techniques, approaching the performance level of Gemini-Ultra and GPT-4. Self-consistency over 64 samples from DeepSeekMath 7B achieves 60.9% on MATH. The mathematical reasoning capability of DeepSeekMath is attributed to two key factors: First, we harness the significant potential of publicly available web data through a meticulously engineered data selection pipeline. Second, we introduce Group Relative Policy Optimization (GRPO), a variant of Proximal Policy Optimization (PPO), that enhances mathematical reasoning abilities while concurrently optimizing the memory usage of PPO.

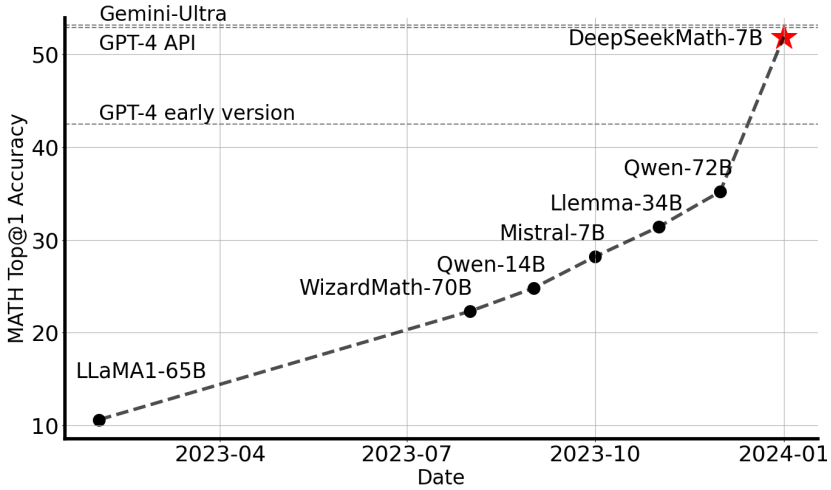


Figure 1 | Top1 accuracy of open-source models on the competition-level MATH benchmark (Hendrycks et al., 2021) without the use of external toolkits and voting techniques.

* Core contributors.
† Work done during internship at DeepSeek-AI.

1. Introduction

Large language models (LLM) have revolutionized the approach to mathematical reasoning in artificial intelligence, spurring significant advancements in both the quantitative reasoning benchmark (Hendrycks et al., 2021) and the geometry reasoning benchmark (Trinh et al., 2024). Moreover, these models have proven instrumental in assisting humans in solving complex mathematical problems (Tao, 2023). However, cutting-edge models such as GPT-4 (OpenAI, 2023) and Gemini-Ultra (Anil et al., 2023) are not publicly available, and the currently accessible open-source models considerably trail behind in performance.

In this study, we introduce DeepSeekMath, a domain-specific language model that significantly outperforms the mathematical capabilities of open-source models and approaches the performance level of GPT-4 on academic benchmarks. To achieve this, we create the DeepSeek-Math Corpus, a large-scale high-quality pre-training corpus comprising 120B math tokens. This dataset is extracted from the Common Crawl (CC) using a fastText-based classifier (Joulin et al., 2016). In the initial iteration, the classifier is trained using instances from OpenWebMath (Paster et al., 2023) as positive examples, while incorporating a diverse selection of other web pages to serve as negative examples. Subsequently, we employ the classifier to mine additional positive instances from the CC, which are further refined through human annotation. The classifier is then updated with this enhanced dataset to improve its performance. The evaluation results indicate that the large-scale corpus is of high quality, as our base model DeepSeekMath-Base 7B achieves 64.2% on GSM8K (Cobbe et al., 2021) and 36.2% on the competition-level MATH dataset (Hendrycks et al., 2021), outperforming Minerva 540B (Lewkowycz et al., 2022a). In addition, the DeepSeekMath Corpus is multilingual, so we notice an improvement in Chinese mathematical benchmarks (Wei et al., 2023; Zhong et al., 2023). We believe that our experience in mathematical data processing is a starting point for the research community, and there is significant room for improvement in the future.

DeepSeekMath-Base is initialized with DeepSeek-Coder-Base-v1.5 7B (Guo et al., 2024), as we notice that starting from a code training model is a better choice compared to a general LLM. Furthermore, we observe the math training also improves model capability on MMLU (Hendrycks et al., 2020) and BBH benchmarks (Suzgun et al., 2022), indicating it does not only enhance the model's mathematical abilities but also amplifies general reasoning capabilities.

After pre-training, we apply mathematical instruction tuning to DeepSeekMath-Base with chain-of-thought (Wei et al., 2022), program-of-thought (Chen et al., 2022; Gao et al., 2023), and tool-integrated reasoning (Gou et al., 2023) data. The resulting model DeepSeekMath-Instruct 7B beats all 7B counterparts and is comparable with 70B open-source instruction-tuned models.

Furthermore, we introduce the Group Relative Policy Optimization (GRPO), a variant reinforcement learning (RL) algorithm of Proximal Policy Optimization (PPO) (Schulman et al., 2017). GRPO foregoes the critic model, instead estimating the baseline from group scores, significantly reducing training resources. By solely using a subset of English instruction tuning data, GRPO obtains a substantial improvement over the strong DeepSeekMath-Instruct, including both in-domain (GSM8K: 82.9% → 88.2%, MATH: 46.8% → 51.7%) and out-of-domain mathematical tasks (e.g., CMATH: 84.6% → 88.8%) during the reinforcement learning phase. We also provide a unified paradigm to understand different methods, such as Rejection Sampling Fine-Tuning (RFT) (Yuan et al., 2023a), Direct Preference Optimization (DPO) (Rafailov et al., 2023), PPO and GRPO. Based on such a unified paradigm, we find that all these methods are conceptualized as either direct or simplified RL techniques. We also conduct extensive experiments, e.g., online v.s. offline training, outcome v.s. process supervision, single-turn v.s. iterative RL and so on,

to deeply investigate the essential elements of this paradigm. At last, we explain why our RL boosts the performance of instruction-tuned models, and further summarize potential directions to achieve more effective RL based on this unified paradigm.

1.1. Contributions

Our contribution includes scalable math pre-training, along with the exploration and analysis of reinforcement learning.

Math Pre-Training at Scale

- Our research provides compelling evidence that the publicly accessible Common Crawl data contains valuable information for mathematical purposes. By implementing a meticulously designed data selection pipeline, we successfully construct the DeepSeekMath Corpus, a high-quality dataset of 120B tokens from web pages filtered for mathematical content, which is almost 7 times the size of the math web pages used by Minerva (Lewkowycz et al., 2022a) and 9 times the size of the recently released OpenWebMath (Paster et al., 2023).
- Our pre-trained base model DeepSeekMath-Base 7B achieves comparable performance with Minerva 540B (Lewkowycz et al., 2022a), indicating the number of parameters is not the only key factor in mathematical reasoning capability. A smaller model pre-trained on high-quality data could achieve strong performance as well.
- We share our findings from math training experiments. Code training prior to math training improves models' ability to solve mathematical problems both with and without tool use. This offers a partial answer to the long-standing question: *does code training improve reasoning abilities?* We believe it does, at least for mathematical reasoning.
- Although training on arXiv papers is common, especially in many math-related papers, it brings no notable improvements on all mathematical benchmarks adopted in this paper.

Exploration and Analysis of Reinforcement Learning

- We introduce Group Relative Policy Optimization (GRPO), an efficient and effective reinforcement learning algorithm. GRPO foregoes the critic model, instead estimating the baseline from group scores, significantly reducing training resources compared to Proximal Policy Optimization (PPO).
- We demonstrate that GRPO significantly enhances the performance of our instruction-tuned model DeepSeekMath-Instruct, by solely using the instruction-tuning data. Furthermore, we observe enhancements in the out-of-domain performance during the reinforcement learning process.
- We provide a unified paradigm to understand different methods, such as RFT, DPO, PPO, and GRPO. We also conduct extensive experiments, e.g., online v.s. offline training, outcome v.s. process supervision, single-turn v.s. iterative reinforcement learning, and so on to deeply investigate the essential elements of this paradigm.
- Based on our unified paradigm, we explore the reasons behind the effectiveness of reinforcement learning, and summarize several potential directions to achieve more effective reinforcement learning of LLMs.

1.2. Summary of Evaluations and Metrics

- **English and Chinese Mathematical Reasoning**: We conduct comprehensive assessments of our models on English and Chinese benchmarks, covering mathematical problems

from grade-school level to college level. English benchmarks include GSM8K (Cobbe et al., 2021), MATH (Hendrycks et al., 2021), SAT (Azerbayev et al., 2023), OCW Courses (Lewkowycz et al., 2022a), MMLU-STEM (Hendrycks et al., 2020). Chinese benchmarks include MGSM-zh (Shi et al., 2023), CMATH (Wei et al., 2023), Gaokao-MathCloze (Zhong et al., 2023), and Gaokao-MathQA (Zhong et al., 2023). We evaluate models' ability to generate self-contained text solutions without tool use, and also the ability to solve problems using Python.

On English benchmarks, DeepSeekMath-Base is competitive with the closed-source Minerva 540B (Lewkowycz et al., 2022a), and surpasses all open-source base models (e.g., Mistral 7B (Jiang et al., 2023) and Llemma-34B (Azerbayev et al., 2023)), regardless of whether they've undergone math pre-training or not, often by a significant margin. Notably, DeepSeekMath-Base is superior on Chinese benchmarks, likely because we don't follow previous works (Azerbayev et al., 2023; Lewkowycz et al., 2022a) to collect English-only math pre-training data, and also include high-quality non-English ones. With mathematical instruction tuning and reinforcement learning, the resulting DeepSeekMath-Instruct and DeepSeekMath-RL demonstrate strong performance, obtaining an accuracy of over 50% on the competition-level MATH dataset for the first time within the open-source community.

- **Formal Mathematics**: We evaluate DeepSeekMath-Base using the informal-to-formal theorem proving task from (Jiang et al., 2022) on miniF2F (Zheng et al., 2021) with Isabelle (Wenzel et al., 2008) chosen to be the proof assistant. DeepSeekMath-Base demonstrates strong few-shot autoformalization performance.
- **Natural Language Understanding, Reasoning, and Code**: To build a comprehensive profile of models' general understanding, reasoning, and coding capabilities, we evaluate DeepSeekMath-Base on the Massive Multitask Language Understanding (MMLU) benchmark (Hendrycks et al., 2020) which encompasses 57 multiple-choice tasks covering diverse subjects, BIG-Bench Hard (BBH) (Suzgun et al., 2022) which consists of 23 challenging tasks that mostly require multi-step reasoning to solve, as well as HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021) which are widely used to evaluate code language models. Math pre-training benefits both language understanding and reasoning performance.

2. Math Pre-Training

2.1. Data Collection and Decontamination

In this section, we will outline the process of constructing the DeepSeekMath Corpus from Common Crawl. As depicted in Figure 2, we present an iterative pipeline that demonstrates how to systematically gather a large-scale mathematical corpus from Common Crawl, starting with a seed corpus (e.g., a small but high-quality collection of math-related dataset). It's worth noting that this approach is also applicable to other domains, such as coding.

First, we choose OpenWebMath (Paster et al., 2023), a collection of high-quality mathematical web texts, as our initial seed corpus. Using this corpus, we train a fastText model (Joulin et al., 2016) to recall more OpenWebMath-like mathematical web pages. Specifically, we randomly select 500,000 data points from the seed corpus as positive training examples and another 500,000 web pages from Common Crawl as negative ones. We employ an open-source library[1] for training, configuring the vector dimension to 256, learning rate to 0.1, the maximum length

[1] `https://fasttext.cc`

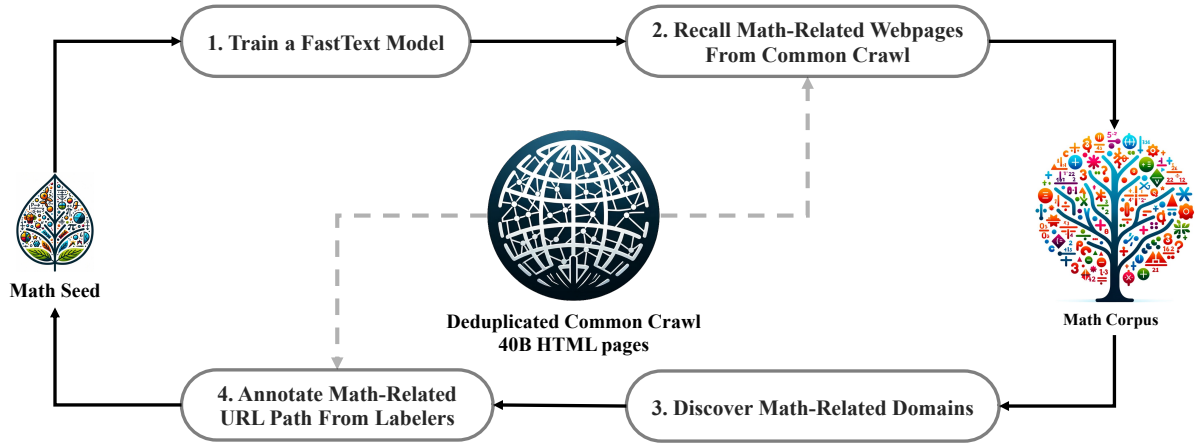


Figure 2 | An iterative pipeline that collects mathematical web pages from Common Crawl.

of word n-gram to 3, the minimum number of word occurrences to 3, and the number of training epochs to 3. To reduce the size of the original Common Crawl, we employ URL-based deduplication and near-deduplication techniques, resulting in 40B HTML web pages. We then recall mathematical web pages from deduplicated Common Crawl with the fastText model. To filter out low-quality mathematical content, we rank the collected pages according to their scores predicted by the fastText model, and only preserve the top-ranking ones. The volume of data preserved is assessed through pre-training experiments on the top 40B, 80B, 120B, and 160B tokens. In the first iteration, we choose to keep the top 40B tokens.

After the first iteration of data collection, numerous mathematical web pages remain uncollected, mainly because the fastText model is trained on a set of positive examples that lacks sufficient diversity. We therefore identify additional mathematical web sources to enrich the seed corpus, so that we can optimize the fastText model. Specifically, we first organize the entire Common Crawl into disjoint domains; a domain is defined as web pages sharing the same base URL. For each domain, we calculate the percentage of web pages that are collected in the first iteration. Domains where over 10% of the web pages have been collected are classified as math-related (e.g., `mathoverflow.net`). Subsequently, we manually annotate the URLs associated with mathematical content within these identified domains (e.g., `mathoverflow.net/questions`). Web pages linked to these URLs, yet uncollected, will be added to the seed corpus. This approach enables us to gather more positive examples, thereby training an improved fastText model capable of recalling more mathematical data in the subsequent iteration. After four iterations of data collection, we end up with 35.5M mathematical web pages, totaling 120B tokens. In the fourth iteration, we notice that nearly 98% of the data has already been collected in the third iteration, so we decide to cease data collection.

To avoid benchmark contamination, we follow Guo et al. (2024) to filter out web pages containing questions or answers from English mathematical benchmarks such as GSM8K (Cobbe et al., 2021) and MATH (Hendrycks et al., 2021) and Chinese benchmarks such as CMATH (Wei et al., 2023) and AGIEval (Zhong et al., 2023). The filtering criteria are as follows: any text segment containing a 10-gram string that matches exactly with any sub-string from the evaluation benchmarks is removed from our math training corpus. For benchmark texts that are shorter than 10 grams but have at least 3 grams, we employ exact matching to filter out contaminated web pages.

2.2. Validating the Quality of the DeepSeekMath Corpus

We run pre-training experiments to investigate how the DeepSeekMath Corpus is compared with the recently released math-training corpora:

- **MathPile** (Wang et al., 2023c): a multi-source corpus (8.9B tokens) aggregated from textbooks, Wikipedia, ProofWiki, CommonCrawl, StackExchange, and arXiv, with the majority (over 85%) sourced from arXiv;
- **OpenWebMath** (Paster et al., 2023): CommonCrawl data filtered for mathematical content, totaling 13.6B tokens;
- **Proof-Pile-2** (Azerbayev et al., 2023): a mathematical corpus consisting of OpenWebMath, AlgebraicStack (10.3B tokens of mathematical code), and arXiv papers (28.0B tokens). When experimenting on Proof-Pile-2, we follow Azerbayev et al. (2023) to use an arXiv:Web:Code ratio of 2:4:1.

2.2.1. Training Setting

We apply math training to a general pre-trained language model with 1.3B parameters, which shares the same framework as the DeepSeek LLMs (DeepSeek-AI, 2024), denoted as DeepSeek-LLM 1.3B. We separately train a model on each mathematical corpus for 150B tokens. All experiments are conducted using the efficient and light-weight HAI-LLM (High-flyer, 2023) training framework. Following the training practice of DeepSeek LLMs, we use the AdamW optimizer (Loshchilov and Hutter, 2017) with $\beta_1 = 0.9$, $\beta_2 = 0.95$, and weight_decay = 0.1, along with a multi-step learning rate schedule where the learning rate reaches the peak after 2,000 warmup steps, decreases to its 31.6% after 80% of the training process, and further decreases to 10.0% of the peak after 90% of the training process. We set the maximum value of learning rate to 5.3e-4, and use a batch size of 4M tokens with a 4K context length.

Math Corpus	Size	English Benchmarks					Chinese Benchmarks		
		GSM8K	MATH	OCW	SAT	MMLU STEM	CMATH	Gaokao MathCloze	Gaokao MathQA
No Math Training	N/A	2.9%	3.0%	2.9%	15.6%	19.5%	12.3%	0.8%	17.9%
MathPile	8.9B	2.7%	3.3%	2.2%	12.5%	15.7%	1.2%	0.0%	2.8%
OpenWebMath	13.6B	11.5%	8.9%	3.7%	31.3%	29.6%	16.8%	0.0%	14.2%
Proof-Pile-2	51.9B	14.3%	11.2%	3.7%	43.8%	29.2%	19.9%	5.1%	11.7%
DeepSeekMath Corpus	**120.2B**	**23.8%**	**13.6%**	**4.8%**	**56.3%**	**33.1%**	**41.5%**	**5.9%**	**23.6%**

Table 1 | Performance of DeepSeek-LLM 1.3B trained on different mathematical corpora, evaluated using few-shot chain-of-thought prompting. Corpus sizes are calculated using our tokenizer with a vocabulary size of 100K.

2.2.2. Evaluation Results

The DeepSeekMath Corpus is of high quality, covers multilingual mathematical content, and is the largest in size.

- **High-quality**: We evaluate downstream performance on 8 mathematical benchmarks using few-shot chain-of-thought prompting Wei et al. (2022). As shown in Table 1, there is a clear performance lead of the model trained on the DeepSeekMath Corpus. Figure 3 shows that the model trained on the DeepSeekMath Corpus demonstrates better performance than

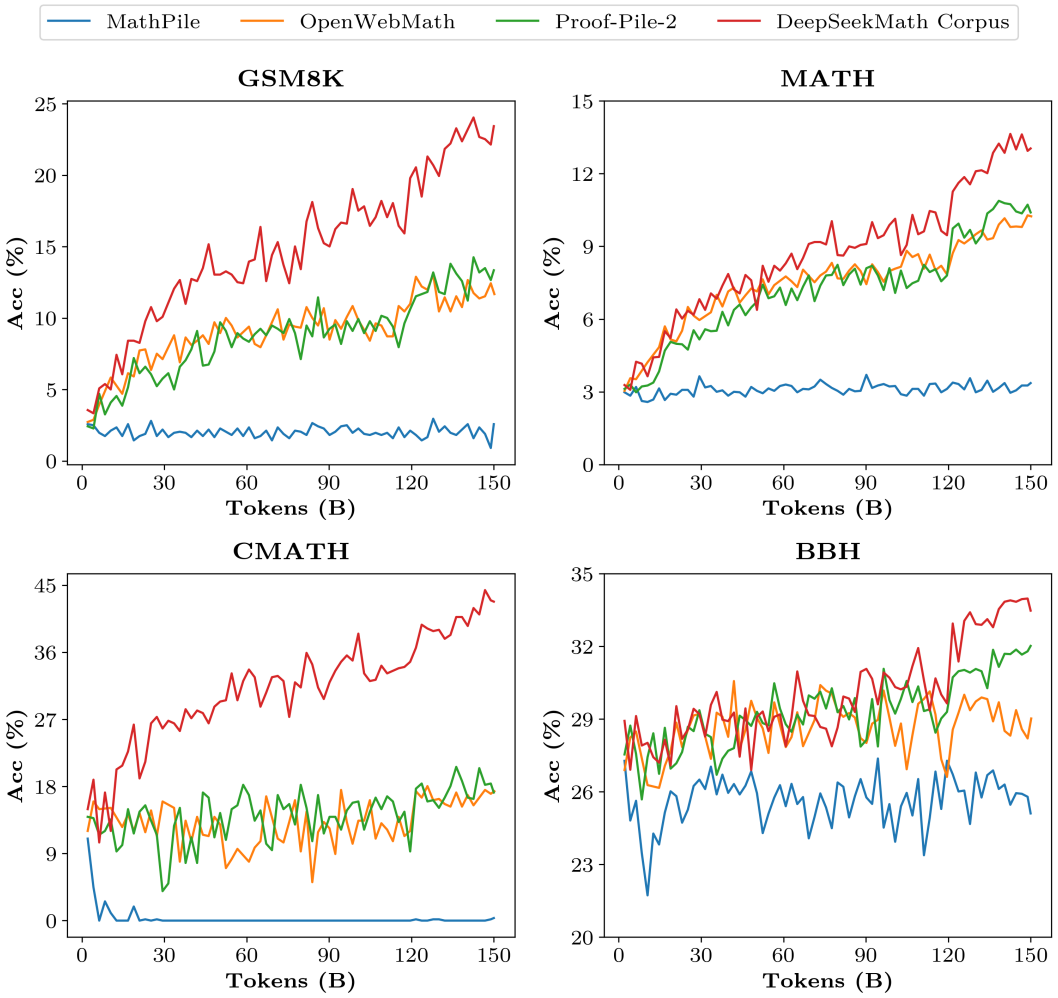


Figure 3 | Benchmark curves of DeepSeek-LLM 1.3B trained on different mathematical corpora.

Proof-Pile-2 at 50B tokens (1 full epoch of Proof-Pile-2), indicating the average quality of DeepSeekMath Corpus is higher.

- **Multilingual**: The DeepSeekMath Corpus encompasses data in multiple languages, predominantly featuring English and Chinese as the two most represented languages. As shown in Table 1, training on the DeepSeekMath Corpus enhances mathematical reasoning performance in both English and Chinese. In contrast, existing mathematical corpora, which are primarily English-centric, show limited improvement and may even hinder performance in Chinese mathematical reasoning.
- **Large-scale**: The DeepSeekMath Corpus is several times larger than existing mathematical corpora. As depicted in Figure 3, DeepSeek-LLM 1.3B, when trained on the DeepSeekMath Corpus, shows a steeper learning curve along with more lasting improvements. In contrast, the baseline corpora are much smaller, and have already been repeated multiple rounds during training, with the resulting model performance quickly reaching a plateau.

2.3. Training and Evaluating DeepSeekMath-Base 7B

In this section, we introduce DeepSeekMath-Base 7B, a base model with strong reasoning abilities, especially in mathematics. Our model is initialized with DeepSeek-Coder-Base-v1.5 7B

(Guo et al., 2024) and trained for 500B tokens. The distribution of the data is as follows: 56% is from the DeepSeekMath Corpus, 4% from AlgebraicStack, 10% from arXiv, 20% is Github code, and the remaining 10% is natural language data from Common Crawl in both English and Chinese. We mainly adopt the training setting specified in Section 2.2.1, except that we set the maximum value of the learning rate to 4.2e-4 and use a batch size of 10M tokens.

We conduct a comprehensive assessment of the mathematical capabilities of DeepSeekMath-Base 7B, focusing on its ability to produce self-contained mathematical solutions without relying on external tools, solve mathematical problems using tools, and conduct formal theorem proving. Beyond mathematics, we also provide a more general profile of the base model, including its performance of natural language understanding, reasoning, and programming skills.

Mathematical Problem Solving with Step-by-Step Reasoning We evaluate DeepSeekMath-Base's performance of solving mathematical problems using few-shot chain-of-thought prompting (Wei et al., 2022), across eight benchmarks in English and Chinese. These benchmarks encompass quantitative reasoning (e.g., GSM8K (Cobbe et al., 2021), MATH (Hendrycks et al., 2021), and CMATH (Wei et al., 2023)) and multiple-choice problems (e.g., MMLU-STEM (Hendrycks et al., 2020) and Gaokao-MathQA (Zhong et al., 2023)), covering diverse fields of mathematics from elementary to college-level complexity.

As shown in Table 2, DeepSeekMath-Base 7B leads in performance across all eight benchmarks among the open-source base models (including the widely-used general model Mistral 7B (Jiang et al., 2023) and the recently released Llemma 34B (Azerbayev et al., 2023) which underwent math training on Proof-Pile-2 (Azerbayev et al., 2023)). Notably, on the competition-level MATH dataset, DeepSeekMath-Base surpasses existing open-source base models by over 10% absolute, and outperforms Minerva 540B (Lewkowycz et al., 2022a), a closed-source base model 77 times larger which builds on PaLM (Lewkowycz et al., 2022b) and is further trained on mathematical texts.

Model	Size	English Benchmarks					Chinese Benchmarks		
		GSM8K	MATH	OCW	SAT	MMLU STEM	CMATH	Gaokao MathCloze	Gaokao MathQA
Closed-Source Base Model									
Minerva	7B	16.2%	14.1%	7.7%	-	35.6%	-	-	-
Minerva	62B	52.4%	27.6%	12.0%	-	53.9%	-	-	-
Minerva	540B	58.8%	33.6%	17.6%	-	63.9%	-	-	-
Open-Source Base Model									
Mistral	7B	40.3%	14.3%	9.2%	71.9%	51.1%	44.9%	5.1%	23.4%
Llemma	7B	37.4%	18.1%	6.3%	59.4%	43.1%	43.4%	11.9%	23.6%
Llemma	34B	54.0%	25.3%	10.3%	71.9%	52.9%	56.1%	11.9%	26.2%
DeepSeekMath-Base	7B	**64.2%**	**36.2%**	**15.4%**	**84.4%**	**56.5%**	**71.7%**	**20.3%**	**35.3%**

Table 2 | Comparisons between DeepSeekMath-Base 7B and strong base models on English and Chinese mathematical benchmarks. Models are evaluated with chain-of-thought prompting. Minerva results are quoted from Lewkowycz et al. (2022a).

Mathematical Problem Solving with Tool Use We evaluate program-aided mathematical reasoning on GSM8K and MATH using few-shot program-of-thought prompting (Chen et al., 2022; Gao et al., 2023). Models are prompted to solve each problem by writing a Python program where libraries such as *math* and *sympy* can be utilized for intricate computations. The execution result of the program is evaluated as the answer. As shown in Table 3, DeepSeekMath-Base 7B outperforms the prior state-of-the-art Llemma 34B.

Model	Size	Problem Solving w/ Tools		Informal-to-Formal Proving	
		GSM8K+Python	MATH+Python	miniF2F-valid	miniF2F-test
Mistral	7B	48.5%	18.2%	18.9%	18.0%
CodeLlama	7B	27.1%	17.2%	16.3%	17.6%
CodeLlama	34B	52.7%	23.5%	18.5%	18.0%
Llemma	7B	41.0%	18.6%	20.6%	22.1%
Llemma	34B	64.6%	26.3%	21.0%	21.3%
DeepSeekMath-Base	7B	**66.9%**	**31.4%**	**25.8%**	**24.6%**

Table 3 | Few-shot evaluation of base models' ability to solve mathematical problems using tools and the ability to conduct informal-to-formal theorem proving in Isabelle.

Formal Mathematics Formal proof automation is beneficial to ensure the accuracy and reliability of mathematical proofs and enhance efficiency, with increasing attention in recent years. We evaluate DeepSeekMath-Base 7B on the task of informal-to-formal proving from (Jiang et al., 2022) which is to generate a formal proof based on an informal statement, a formal counterpart of the statement, and an informal proof. We evaluate on miniF2F (Zheng et al., 2021), a benchmark for formal Olympiad-level mathematics, and generate a formal proof in Isabelle for each problem with few-shot prompting. Following Jiang et al. (2022), we leverage models to generate proof sketches, and execute the off-the-shelf automated prover Sledgehammer (Paulson, 2010) to fill in the missing details. As shown in Table 3, DeepSeekMath-Base 7B demonstrates strong performance in proof autoformalization.

Model	Size	MMLU	BBH	HumanEval (Pass@1)	MBPP (Pass@1)
Mistral	7B	**62.4%**	55.7%	28.0%	41.4%
DeepSeek-Coder-Base-v1.5†	7B	42.9%	42.9%	40.2%	52.6%
DeepSeek-Coder-Base-v1.5	7B	49.1%	55.2%	**43.2%**	**60.4%**
DeepSeekMath-Base	7B	54.9%	**59.5%**	40.9%	52.6%

Table 4 | Evaluation on natural language understanding, reasoning, and code benchmarks. DeepSeek-Coder-Base-v1.5† is the checkpoint right before learning rate decay, which is used to train DeepSeekMath-Base. On MMLU and BBH, we use few-shot chain-of-thought prompting. On HumanEval and MBPP, we evaluate model performance under the zero-shot setting and a few-shot setting, respectively.

Natural Language Understanding, Reasoning, and Code We evaluate model performance of natural language understanding on MMLU (Hendrycks et al., 2020), reasoning on BBH (Suzgun et al., 2022), and coding capabilities on HumanEval (Chen et al., 2021) and MBPP (Austin et al.,

2021). As shown in Table 4, DeepSeekMath-Base 7B exhibits significant enhancements in performance on MMLU and BBH over its precursor, DeepSeek-Coder-Base-v1.5 (Guo et al., 2024), illustrating the positive impact of math training on language understanding and reasoning. Additionally, by including code tokens for continual training, DeepSeekMath-Base 7B effectively maintains the performance of DeepSeek-Coder-Base-v1.5 on the two coding benchmarks. Overall, DeepSeekMath-Base 7B significantly outperforms the general model Mistral 7B (Jiang et al., 2023) on the three reasoning and coding benchmarks.

3. Supervised Fine-Tuning

3.1. SFT Data Curation

We construct a mathematical instruction-tuning dataset covering English and Chinese problems from different mathematical fields and of varying complexity levels: problems are paired with solutions in chain-of-thought (CoT) (Wei et al., 2022), program-of-thought (PoT) (Chen et al., 2022; Gao et al., 2023), and tool-integrated reasoning format (Gou et al., 2023). The total number of training examples is 776K.

- **English mathematical datasets**: We annotate GSM8K and MATH problems with tool-integrated solutions, and adopt a subset of MathInstruct (Yue et al., 2023) along with the training set of Lila-OOD (Mishra et al., 2022) where problems are solved with CoT or PoT. Our English collection covers diverse fields of mathematics, e.g., algebra, probability, number theory, calculus, and geometry.
- **Chinese mathematical datasets**: We collect Chinese K-12 mathematical problems spanning 76 sub-topics such as linear equations, with solutions annotated in both CoT and tool-integrated reasoning format.

3.2. Training and Evaluating DeepSeekMath-Instruct 7B

In this section, we introduce DeepSeekMath-Instruct 7B which undergoes mathematical instruction tuning based on DeepSeekMath-Base. Training examples are randomly concatenated until reaching a maximum context length of 4K tokens. We train the model for 500 steps with a batch size of 256 and a constant learning rate of 5e-5.

We evaluate models' mathematical performance both without and with tool use, on 4 quantitative reasoning benchmarks in English and Chinese. We benchmark our model against the leading models of the time:

- **Closed-source models** include: (1) the GPT family among which GPT-4 (OpenAI, 2023) and GPT-4 Code Interpreter [2] are the most capable ones, (2) Gemini Ultra and Pro (Anil et al., 2023), (3) Inflection-2 (Inflection AI, 2023), (4) Grok-1 [3], as well as models recently released by Chinese companies including (5) Baichuan-3 [4], (6) the latest GLM-4 [5] from the GLM family (Du et al., 2022). These models are for general purposes, most of which have undergone a series of alignment procedures.
- **Open-source models** include: general models like (1) DeepSeek-LLM-Chat 67B (DeepSeek-AI, 2024), (2) Qwen 72B (Bai et al., 2023), (3) SeaLLM-v2 7B (Nguyen et al., 2023), and (4)

[2] https://openai.com/blog/chatgpt-plugins#code-interpreter
[3] https://x.ai/model-card
[4] https://www.baichuan-ai.com
[5] https://open.bigmodel.cn/dev/api#glm-4

ChatGLM3 6B (ChatGLM3 Team, 2023), as well as models with enhancements in mathematics including (5) InternLM2-Math 20B [6] which builds on InternLM2 and underwent math training followed by instruction tuning, (6) Math-Shepherd-Mistral 7B which applys PPO training (Schulman et al., 2017) to Mistral 7B (Jiang et al., 2023) with a process-supervised reward model, (7) the WizardMath series (Luo et al., 2023) which improves mathematical reasoning in Mistral 7B and Llama-2 70B (Touvron et al., 2023) using evolve-instruct (i.e., a version of instruction tuning that uses AI-evolved instructions) and PPO training with training problems primarily sourced from GSM8K and MATH, (8) MetaMath 70B (Yu et al., 2023) which is Llama-2 70B fine-tuned on an augmented version of GSM8K and MATH, (9) ToRA 34B Gou et al. (2023) which is CodeLlama 34B fine-tuned to do tool-integrated mathematical reasoning, (10) MAmmoTH 70B (Yue et al., 2023) which is Llama-2 70B instruction-tuned on MathInstruct.

As shown in Table 5, under the evaluation setting where tool use is disallowed, DeepSeekMath-Instruct 7B demonstrates strong performance of step-by-step reasoning. Notably, on the competition-level MATH dataset, our model surpasses all open-source models and the majority of proprietary models (e.g., Inflection-2 and Gemini Pro) by at least 9% absolute. This is true even for models that are substantially larger (e.g., Qwen 72B) or have been specifically enhanced through math-focused reinforcement learning (e.g., WizardMath-v1.1 7B). While DeepSeekMath-Instruct rivals the Chinese proprietary models GLM-4 and Baichuan-3 on MATH, it still underperforms GPT-4 and Gemini Ultra.

Under the evaluation setting where models are allowed to integrate natural language reasoning and program-based tool use for problem solving, DeepSeekMath-Instruct 7B approaches an accuracy of 60% on MATH, surpassing all existing open-source models. On the other benchmarks, our model is competitive with DeepSeek-LLM-Chat 67B, the prior state-of-the-art that is 10 times larger.

4. Reinforcement Learning

4.1. Group Relative Policy Optimization

Reinforcement learning (RL) has been proven to be effective in further improving the mathematical reasoning ability of LLMs after the Supervised Fine-Tuning (SFT) stage (Luo et al., 2023; Wang et al., 2023b). In this section, we introduce our efficient and effective RL algorithm, Group Relative Policy Optimization (GRPO).

4.1.1. From PPO to GRPO

Proximal Policy Optimization (PPO) (Schulman et al., 2017) is an actor-critic RL algorithm that is widely used in the RL fine-tuning stage of LLMs (Ouyang et al., 2022). In particular, it optimizes LLMs by maximizing the following surrogate objective:

$$\mathcal{J}_{PPO}(\theta) = \mathbb{E}[q \sim P(Q), o \sim \pi_{\theta_{old}}(O|q)] \frac{1}{|o|} \sum_{t=1}^{|o|} \min \left[\frac{\pi_\theta(o_t|q,o_{<t})}{\pi_{\theta_{old}}(o_t|q,o_{<t})} A_t, \text{clip}\left(\frac{\pi_\theta(o_t|q,o_{<t})}{\pi_{\theta_{old}}(o_t|q,o_{<t})}, 1-\varepsilon, 1+\varepsilon \right) A_t \right], \quad (1)$$

where π_θ and $\pi_{\theta_{old}}$ are the current and old policy models, and q, o are questions and outputs sampled from the question dataset and the old policy $\pi_{\theta_{old}}$, respectively. ε is a clipping-related hyper-parameter introduced in PPO for stabilizing training. A_t is the advantage, which is computed by applying Generalized Advantage Estimation (GAE) (Schulman et al., 2015), based

[6] https://github.com/InternLM/InternLM-Math

Model	Size	English Benchmarks		Chinese Benchmarks	
		GSM8K	MATH	MGSM-zh	CMATH
Chain-of-Thought Reasoning					
Closed-Source Model					
Gemini Ultra	-	94.4%	53.2%	-	-
GPT-4	-	92.0%	52.9%	-	86.0%
Inflection-2	-	81.4%	34.8%	-	-
GPT-3.5	-	80.8%	34.1%	-	73.8%
Gemini Pro	-	86.5%	32.6%	-	-
Grok-1	-	62.9%	23.9%	-	-
Baichuan-3	-	88.2%	49.2%	-	-
GLM-4	-	87.6%	47.9%	-	-
Open-Source Model					
InternLM2-Math	20B	82.6%	37.7%	-	-
Qwen	72B	78.9%	35.2%	-	-
Math-Shepherd-Mistral	7B	84.1%	33.0%	-	-
WizardMath-v1.1	7B	83.2%	33.0%	-	-
DeepSeek-LLM-Chat	67B	84.1%	32.6%	74.0%	80.3%
MetaMath	70B	82.3%	26.6%	66.4%	70.9%
SeaLLM-v2	7B	78.2%	27.5%	64.8%	-
ChatGLM3	6B	72.3%	25.7%	-	-
WizardMath-v1.0	70B	81.6%	22.7%	64.8%	65.4%
DeepSeekMath-Instruct	7B	82.9%	46.8%	73.2%	84.6%
DeepSeekMath-RL	7B	**88.2%**	**51.7%**	**79.6%**	**88.8%**
Tool-Integrated Reasoning					
Closed-Source Model					
GPT-4 Code Interpreter	-	97.0%	69.7%	-	-
Open-Source Model					
InternLM2-Math	20B	80.7%	54.3%	-	-
DeepSeek-LLM-Chat	67B	86.7%	51.1%	76.4%	85.4%
ToRA	34B	80.7%	50.8%	41.2%	53.4%
MAmmoTH	70B	76.9%	41.8%	-	-
DeepSeekMath-Instruct	7B	83.7%	57.4%	72.0%	84.3%
DeepSeekMath-RL	7B	**86.7%**	**58.8%**	**78.4%**	**87.6%**

Table 5 | Performance of Open- and Closed-Source models with both Chain-of-Thought and Tool-Integrated Reasoning on English and Chinese Benchmarks. Scores in gray denote majority votes with 32 candidates; The others are Top1 scores. DeepSeekMath-RL 7B beats all open-source models from 7B to 70B, as well as the majority of closed-source models. Although DeepSeekMath-RL 7B is only further trained on chain-of-thought-format instruction tuning data of GSM8K and MATH, it improves over DeepSeekMath-Instruct 7B on all benchmarks.

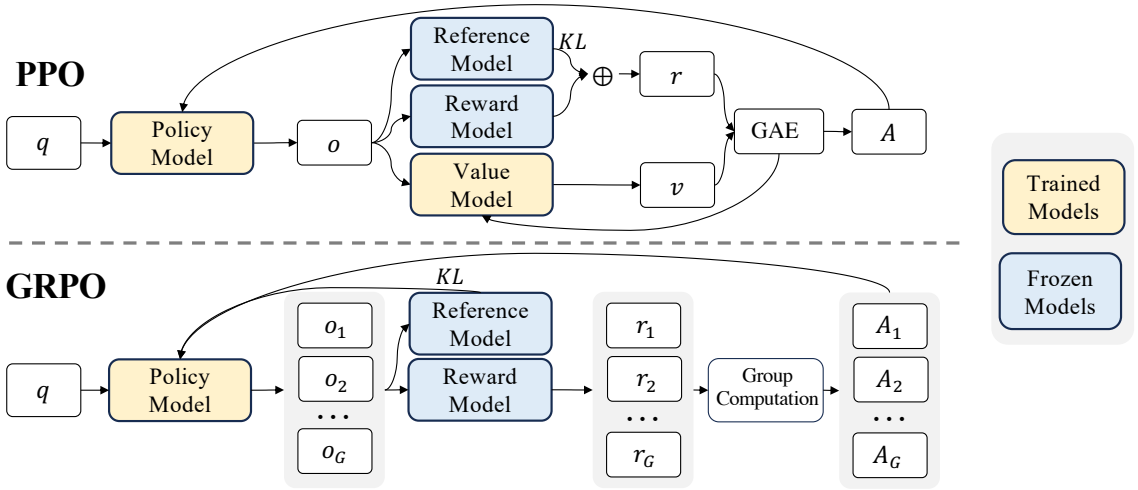


Figure 4 | Demonstration of PPO and our GRPO. GRPO foregoes the value model, instead estimating the baseline from group scores, significantly reducing training resources.

on the rewards $\{r_{\geq t}\}$ and a learned value function V_ψ. Thus, in PPO, a value function needs to be trained alongside the policy model and to mitigate over-optimization of the reward model, the standard approach is to add a per-token KL penalty from a reference model in the reward at each token (Ouyang et al., 2022), i.e.,

$$r_t = r_\varphi(q, o_{\leq t}) - \beta \log \frac{\pi_\theta(o_t|q, o_{<t})}{\pi_{ref}(o_t|q, o_{<t})}, \tag{2}$$

where r_φ is the reward model, π_{ref} is the reference model, which is usually the initial SFT model, and β is the coefficient of the KL penalty.

As the value function employed in PPO is typically another model of comparable size as the policy model, it brings a substantial memory and computational burden. Additionally, during RL training, the value function is treated as a baseline in the calculation of the advantage for variance reduction. While in the LLM context, usually only the last token is assigned a reward score by the reward model, which may complicate the training of a value function that is accurate at each token. To address this, as shown in Figure 4, we propose Group Relative Policy Optimization (GRPO), which obviates the need for additional value function approximation as in PPO, and instead uses the average reward of multiple sampled outputs, produced in response to the same question, as the baseline. More specifically, for each question q, GRPO samples a group of outputs $\{o_1, o_2, \cdots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)]$$
$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}\hat{A}_{i,t}, \text{clip}\left(\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}, 1-\varepsilon, 1+\varepsilon\right)\hat{A}_{i,t}\right] - \beta\mathbb{D}_{KL}\left[\pi_\theta||\pi_{ref}\right]\right\}, \tag{3}$$

where ε and β are hyper-parameters, and $\hat{A}_{i,t}$ is the advantage calculated based on relative rewards of the outputs inside each group only, which will be detailed in the following subsections. The group relative way that GRPO leverages to calculate the advantages, aligns well with the comparative nature of rewards models, as reward models are typically trained on datasets of comparisons between outputs on the same question. Also note that, instead of adding KL penalty in the reward, GRPO regularizes by directly adding the KL divergence between the trained policy and the reference policy to the loss, avoiding complicating the calculation of $\hat{A}_{i,t}$.

Algorithm 1 Iterative Group Relative Policy Optimization

Input initial policy model $\pi_{\theta_{\text{init}}}$; reward models r_φ; task prompts $\mathcal{D}$; hyperparameters ε, β, μ

policy model $\pi_\theta \leftarrow \pi_{\theta_{\text{init}}}$
for iteration = 1, ..., I **do**
  reference model $\pi_{ref} \leftarrow \pi_\theta$
  for step = 1, ..., M **do**
    Sample a batch $\mathcal{D}_b$ from $\mathcal{D}$
    Update the old policy model $\pi_{\theta_{old}} \leftarrow \pi_\theta$
    Sample G outputs $\{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(\cdot \mid q)$ for each question $q \in \mathcal{D}_b$
    Compute rewards $\{r_i\}_{i=1}^{G}$ for each sampled output o_i by running r_φ
    Compute $\hat{A}_{i,t}$ for the t-th token of o_i through group relative advantage estimation.
    for GRPO iteration = 1, ..., μ **do**
      Update the policy model π_θ by maximizing the GRPO objective (Equation 21)
  Update r_φ through continuous training using a replay mechanism.

Output π_θ

And different from the KL penalty term used in (2), we estimate the KL divergence with the following unbiased estimator (Schulman, 2020):

$$\mathbb{D}_{KL}\left[\pi_\theta||\pi_{ref}\right] = \frac{\pi_{ref}(o_{i,t}|q,o_{i,<t})}{\pi_\theta(o_{i,t}|q,o_{i,<t})} - \log\frac{\pi_{ref}(o_{i,t}|q,o_{i,<t})}{\pi_\theta(o_{i,t}|q,o_{i,<t})} - 1, \tag{4}$$

which is guaranteed to be positive.

4.1.2. Outcome Supervision RL with GRPO

Formally, for each question q, a group of outputs $\{o_1, o_2, \cdots, o_G\}$ are sampled from the old policy model $\pi_{\theta_{old}}$. A reward model is then used to score the outputs, yielding G rewards $\mathbf{r} = \{r_1, r_2, \cdots, r_G\}$ correspondingly. Subsequently, these rewards are normalized by subtracting the group average and dividing by the group standard deviation. Outcome supervision provides the normalized reward at the end of each output o_i and sets the advantages $\hat{A}_{i,t}$ of all tokens in the output as the normalized reward, i.e., $\hat{A}_{i,t} = \widetilde{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$, and then optimizes the policy by maximizing the objective defined in equation (3).

4.1.3. Process Supervision RL with GRPO

Outcome supervision only provides a reward at the end of each output, which may not be sufficient and efficient to supervise the policy in complex mathematical tasks. Following Wang et al. (2023b), we also explore process supervision, which provides a reward at the end of each reasoning step. Formally, given the question q and G sampled outputs $\{o_1, o_2, \cdots, o_G\}$, a process reward model is used to score each step of the outputs, yielding corresponding rewards: $\mathbf{R} = \{\{r_1^{index(1)}, \cdots, r_1^{index(K_1)}\}, \cdots, \{r_G^{index(1)}, \cdots, r_G^{index(K_G)}\}\}$, where $index(j)$ is the end token index of the j-th step, and K_i is the total number of steps in the i-th output. We also normalize these rewards with the average and the standard deviation, i.e., $\widetilde{r}_i^{index(j)} = \frac{r_i^{index(j)} - \text{mean}(\mathbf{R})}{\text{std}(\mathbf{R})}$. Subsequently, the process supervision calculates the advantage of each token as the sum of the normalized rewards from the following steps, i.e., $\hat{A}_{i,t} = \sum_{index(j)\geq t} \widetilde{r}_i^{index(j)}$, and then optimizes the policy by maximizing the objective defined in equation (3).

4.1.4. Iterative RL with GRPO

As the reinforcement learning training process progresses, the old reward model may not be sufficient to supervise the current policy model. Therefore, we also explore the iterative RL with GRPO. As shown in Algorithm 1, in iterative GRPO, we generate new training sets for the reward model based on the sampling results from the policy model and continually train the old reward model using a replay mechanism that incorporates 10% of historical data. Then, we set the reference model as the policy model, and continually train the policy model with the new reward model.

4.2. Training and Evaluating DeepSeekMath-RL

We conduct RL based on DeepSeekMath-Instruct 7B. The training data of RL are chain-of-thought-format questions related to GSM8K and MATH from the SFT data, which consists of around 144K questions. We exclude other SFT questions to investigate the impact of RL on benchmarks that lack data throughout the RL phase. We construct the training set of reward models following (Wang et al., 2023b). We train our initial reward model based on the DeepSeekMath-Base 7B with a learning rate of 2e-5. For GRPO, we set the learning rate of the policy model as 1e-6. The KL coefficient is 0.04. For each question, we sample 64 outputs. The max length is set to 1024, and the training batch size is 1024. The policy model only has a single update following each exploration stage. We evaluate DeepSeekMath-RL 7B on benchmarks following DeepSeekMath-Instruct 7B. For DeepSeekMath-RL 7B, GSM8K and MATH with chain-of-thought reasoning can be regarded as in-domain tasks and all the other benchmarks can be regarded as out-of-domain tasks.

Table 5 demonstrates the performance of open- and closed-source models with both chain-of-thought and tool-integrated reasoning on English and Chinese benchmarks. We find that: 1) DeepSeekMath-RL 7B attains accuracies of 88.2% and 51.7% on GSM8K and MATH, respectively, utilizing chain-of-thought reasoning. This performance surpasses that of all open-source models in the 7B to 70B range, as well as the majority of closed-source models. 2) Crucially, DeepSeekMath-RL 7B is only trained on chain-of-thought-format instruction tuning data of GSM8K and MATH, starting from DeepSeekMath-Instruct 7B. Despite the constrained scope of its training data, it outperforms DeepSeekMath-Instruct 7B across all evaluation metrics, showcasing the effectiveness of reinforcement learning.

5. Discussion

In this section, we will share our findings in pre-training and RL experiments.

5.1. Lessons Learnt in Pre-Training

We first share our experience in pre-training. Unless otherwise specified, we will adhere to the training settings outlined in Section 2.2.1. It is worth noting that, when referring to the DeepSeekMath Corpus in this section, we use an 89B-token dataset from the second iteration of the data collection process.

5.1.1. Code Training Benefits Mathematical Reasoning

A popular yet unverified hypothesis suggests that code training improves reasoning. We attempt to offer a partial response to this, particularly within the mathematical domain: code training

Training Setting	Training Tokens			w/o Tool Use			w/ Tool Use	
	General	Code	Math	GSM8K	MATH	CMATH	GSM8K+Python	MATH+Python
No Continual Training	–	–	–	2.9%	3.0%	12.3%	2.7%	2.3%
Two-Stage Training								
Stage 1: General Training	400B	–	–	2.9%	3.2%	14.8%	3.3%	2.3%
Stage 2: Math Training	–	–	150B	19.1%	14.4%	37.2%	14.3%	6.7%
Stage 1: Code Training	–	400B	–	5.9%	3.6%	19.9%	12.4%	10.0%
Stage 2: Math Training	–	–	150B	**21.9%**	**15.3%**	**39.7%**	17.4%	9.4%
One-Stage Training								
Math Training	–	–	150B	20.5%	13.1%	37.6%	11.4%	6.5%
Code & Math Mixed Training	–	400B	150B	17.6%	12.1%	36.3%	**19.7%**	**13.5%**

Table 6 | Investigation of how code affects mathematical reasoning under different training settings. We experiment with DeepSeek-LLM 1.3B, and evaluate its mathematical reasoning performance without and with tool use via few-shot chain-of-thought prompting and few-shot program-of-thought prompting, respectively.

improves models' ability to do mathematical reasoning both with and without tool use.

To study how code training affects mathematical reasoning, we experimented with the following two-stage training and one-stage training settings:

Two-Stage Training

- **Code Training for 400B Tokens → Math Training for 150B Tokens**: We train DeepSeek-LLM 1.3B for 400B code tokens followed by 150B math tokens;
- **General Training for 400B Tokens → Math Training for 150B Tokens**: As a control experiment, we also experiment with general tokens (sampled from a large-scale general corpus created by DeepSeek-AI) instead of code tokens in the first stage of training, in an attempt to investigate the advantages of code tokens over general tokens in improving mathematical reasoning.

One-Stage Training

- **Math Training for 150B Tokens**: We train DeepSeek-LLM 1.3B for 150B math tokens;
- **Training on a mixture of 400B Code Tokens and 150B Math Tokens**: Math training following code training degrades coding performance. We investigate whether code tokens, when mixed with math tokens for one-stage training, would still improve mathematical reasoning and also alleviate the problem of catastrophic forgetting.

Results Table 6 and Table 7 demonstrate the downstream performance under different training settings.

Code training benefits program-aided mathematical reasoning, both under the two-stage training and one-stage training settings. As shown in Table 6, under the two-stage training setting, code training alone already significantly enhances the ability to solve GSM8K and MATH problems using Python. Math training in the second stage yields further improvements. Interestingly, under the one-stage training setting, mixing code tokens and math tokens effectively mitigates the issue of catastrophic forgetting that arises from two-stage training, and also synergizes coding (Table 7) and program-aided mathematical reasoning (Table 6).

Training Setting	Training Tokens			MMLU	BBH	HumanEval (Pass@1)	MBPP (Pass@1)
	General	Code	Math				
No Continual Training	–	–	–	24.5%	28.1%	12.2%	13.0%
Two-Stage Training							
Stage 1: General Training	400B	–	–	25.9%	27.7%	15.2%	13.6%
Stage 2: Math Training	–	–	150B	33.1%	32.7%	12.8%	13.2%
Stage 1: Code Training	–	400B	–	25.0%	31.5%	25.0%	**40.0%**
Stage 2: Math Training	–	–	150B	**36.2%**	35.3%	12.2%	17.0%
One-Stage Training							
Math Training	–	–	150B	32.3%	32.5%	11.6%	13.2%
Code & Math Mixed Training	–	400B	150B	33.5%	**35.6%**	**29.3%**	39.4%

Table 7 | Investigation of how different settings of code and math training affect model performance of language understanding, reasoning, and coding. We experiment with DeepSeek-LLM 1.3B. We evaluate the models on MMLU and BBH using few-shot chain-of-thought prompting. On HumanEval and MBPP, we conduct zero-shot and few-shot evaluations, respectively.

Model	Size	ArXiv Corpus	English Benchmarks					Chinese Benchmarks		
			GSM8K	MATH	OCW	SAT	MMLU STEM	CMATH	Gaokao MathCloze	Gaokao MathQA
DeepSeek-LLM	1.3B	No Math Training	2.9%	3.0%	2.9%	15.6%	19.5%	12.3%	0.8%	17.9%
		MathPile	2.7%	3.3%	2.2%	12.5%	15.7%	1.2%	0.0%	2.8%
		ArXiv-RedPajama	3.3%	3.4%	4.0%	9.4%	9.0%	7.4%	0.8%	2.3%
DeepSeek-Coder-Base-v1.5	7B	No Math Training	29.0%	12.5%	6.6%	40.6%	38.1%	45.9%	5.9%	21.1%
		MathPile	23.6%	11.5%	7.0%	46.9%	35.8%	37.9%	4.2%	25.6%
		ArXiv-RedPajama	28.1%	11.1%	7.7%	50.0%	35.2%	42.6%	7.6%	24.8%

Table 8 | Effect of math training on different arXiv datasets. Model performance is evaluated with few-shot chain-of-thought prompting.

ArXiv Corpus	miniF2F-valid	miniF2F-test
No Math Training	20.1%	21.7%
MathPile	16.8%	16.4%
ArXiv-RedPajama	14.8%	11.9%

Table 9 | Effect of math training on different arXiv corpora, the base model being DeepSeek-Coder-Base-v1.5 7B. We evaluate informal-to-formal proving in Isabelle.

Code training also improves mathematical reasoning without tool use. Under the two-stage training setting, the initial stage of code training already results in moderate enhancements. It also boosts the efficiency of the subsequent math training, eventually leading to the best performance. However, combining code tokens and math tokens for one-stage training compromises mathematical reasoning without tool use. One conjecture is that DeepSeek-LLM 1.3B, due to its limited scale, lacks the capacity to fully assimilate both code and mathematical data simultaneously.

5.1.2. ArXiv Papers Seem Ineffective in Improving Mathematical Reasoning

ArXiv papers are commonly included as a component of math pre-training data (Azerbayev et al., 2023; Lewkowycz et al., 2022a; Polu and Sutskever, 2020; Wang et al., 2023c). However,

detailed analysis regarding their impact on mathematical reasoning has not been extensively conducted. Perhaps counter-intuitively, according to our experiments, arXiv papers seem ineffective in improving mathematical reasoning. We experiment with models of different sizes, including DeepSeek-LLM 1.3B and DeepSeek-Coder-Base-v1.5 7B (Guo et al., 2024), using arXiv corpora that underwent varied processing pipelines:

- **MathPile** (Wang et al., 2023c): an 8.9B-token corpus developed with cleaning and filtering heuristic rules, over 85% of which are scientific arXiv papers;
- **ArXiv-RedPajama** (Computer, 2023): the entirety of arXiv LaTeX files with preambles, comments, macros, and bibliographies removed, totaling 28.0B tokens.

In our experiments, we separately train DeepSeek-LLM 1.3B for 150B tokens and DeepSeek-Coder-Base-v1.5 7B for 40B tokens on each arXiv corpus. It seems that arXiv papers are ineffective in improving mathematical reasoning. When trained on a arXiv-only corpus, both models display no notable improvements or even deterioration across various mathematical benchmarks of different complexities employed in this study. These benchmarks include quantitative reasoning datasets like GSM8K and MATH (Table 8), multiple-choice challenges like MMLU-STEM (Table 8), and formal mathematics like miniF2F (Table 9).

However, this conclusion has its limitations and should be taken with a grain of salt. We have not yet studied:

- The impact of arXiv tokens on specific math-related tasks not included in this research, such as informalization of theorems which is to convert formal statements or proofs to their informal versions;
- The effect of arXiv tokens when combined with other types of data;
- Whether the benefits of arXiv papers would manifest themselves at a larger model scale.

Thus, further exploration is required, which we leave for future studies.

5.2. Insights of Reinforcement Learning

5.2.1. Towards to a Unified Paradigm

In this section, we provide a unified paradigm to analyze different training methods, such as SFT, RFT, DPO, PPO, GRPO, and further conduct experiments to explore the factors of the unified paradigm. Generally, the gradient with respect to the parameter θ of a training method can be written as:

$$\nabla_\theta \mathcal{J}_{\mathcal{A}}(\theta) = \mathbb{E}[\underbrace{(q,o) \sim \mathcal{D}}_{Data\ Source}] \left(\frac{1}{|o|} \sum_{t=1}^{|o|} \underbrace{GC_{\mathcal{A}}(q,o,t,\pi_{rf})}_{Gradient\ Coefficient} \nabla_\theta \log \pi_\theta(o_t|q,o_{<t}) \right). \tag{5}$$

There exist three key components: 1) *Data Source* $\mathcal{D}$, which determines the training data; 2) *Reward Function* π_{rf}, which is the source of the training reward signal; 3) *Algorithm* $\mathcal{A}$: which processes the training data and the reward signal to the gradient coefficient GC that determines the magnitude of the penalty or reinforcement for the data. We analyze several representative methods based on such a unified paradigm:

- **Supervised Fine-tuning (SFT)**: SFT fine-tunes pretrained model on human selected SFT data.

Methods	Data Source	Reward Function	Gradient Coefficient
SFT	$q, o \sim P_{sft}(Q, O)$	-	1
RFT	$q \sim P_{sft}(Q), o \sim \pi_{sft}(O\|q)$	Rule	Equation 10
DPO	$q \sim P_{sft}(Q), o^+, o^- \sim \pi_{sft}(O\|q)$	Rule	Equation 14
Online RFT	$q \sim P_{sft}(Q), o \sim \pi_\theta(O\|q)$	Rule	Equation 10
PPO	$q \sim P_{sft}(Q), o \sim \pi_\theta(O\|q)$	Model	Equation 18
GRPO	$q \sim P_{sft}(Q), \{o_i\}_{i=1}^{G} \sim \pi_\theta(O\|q)$	Model	Equation 21

Table 10 | The data source and gradient coefficient of different methods. P_{sft} denotes the data distribution of supervised fine-tuning datasets. $\pi_{\theta_{sft}}$ and π_θ denote the supervised fine-tuned model and the real-time policy model during the online training process, respectively.

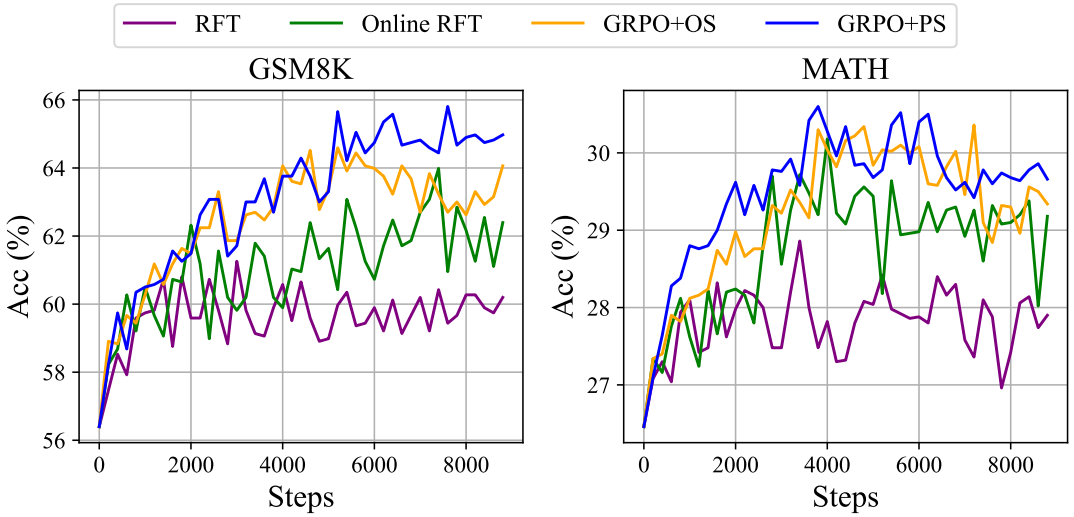


Figure 5 | Performance of the DeepSeekMath-Instruct 1.3B model, which was further trained using various methods, on two benchmarks.

- **Rejection Sampling Fine-tuning (RFT)**: RFT further fine-tunes the SFT model on the filtered outputs sampled from the SFT model based on SFT questions. RFT filters the outputs based on the correctness of their answers.
- **Direct Preference Optimization (DPO)**: DPO further refines the SFT model by fine-tuning it on augmented outputs sampled from the SFT model, using pair-wise DPO loss.
- **Online Rejection Sampling Fine-tuning (Online RFT)**: Different from RFT, Online RFT initiates the policy model using the SFT model and refines it by fine-tuning with the augmented outputs sampled from the real-time policy model.
- **PPO/GRPO**: PPO/GRPO initializes the policy model using the SFT model and reinforces it with the outputs sampled from the real-time policy model.

We summarize the components of these methods in Table 10. Please refer to Appendix A.1 for a more detailed derivation process.

Observation about Data Source We divide the data source into two categories, online sampling, and offline sampling. Online sampling denotes that the training data is from the exploration results of the real-time training policy model, while offline sampling denotes that the

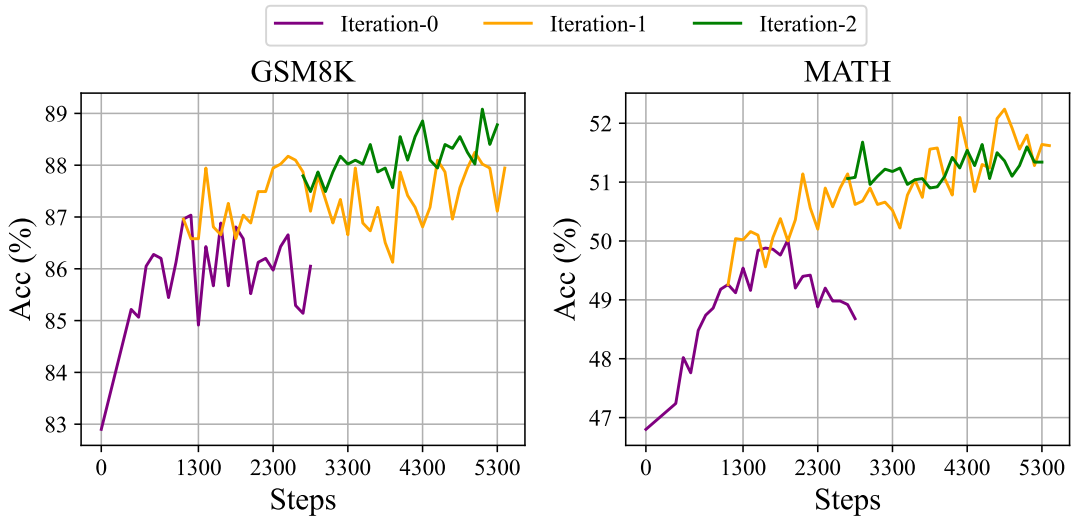


Figure 6 | Performance of iterative reinforcement learning with DeepSeekMath-Instruct 7B on two benchmarks.

training data is from the sampling results of the initial SFT model. RFT and DPO follow the offline style, while Online RFT and GRPO follow the online style.

As shown in Figure 5, we find that the Online RFT significantly outperforms RFT on two benchmarks. Specifically, Online RFT is comparable to RFT in the early stage of training but gains an absolute advantage in the later stage, demonstrating the superiority of online training. This is intuitive, as in the initial stage, the actor and the SFT model exhibit close resemblance, with the sampled data revealing only minor differences. In the later stage, however, the data sampled from the actor will exhibit more significant differences, and real-time data sampling will offer greater advantages.

Observation about Gradient Coefficient The algorithm processes the reward signal to the gradient coefficient to update the model parameter. We divide the reward function as 'Rule' and 'Model' in our experiments. Rule refers to judging the quality of a response based on the correctness of the answer, and Model denotes that we train a reward model to score each response. The training data of the reward model is based on the rule judgment. Equations 10 and 21 highlight a key difference between GRPO and Online RFT: GRPO uniquely adjusts its gradient coefficient based on the reward value provided by the reward model. This allows for differential reinforcement and penalization of responses according to their varying magnitudes. In contrast, Online RFT lacks this feature; it does not penalize incorrect responses and uniformly reinforces all responses with correct answers at the same level of intensity.

As demonstrated in Figure 5, GRPO surpasses online RFT, thereby highlighting the efficiency of altering positive and negative gradient coefficients. In addition, GRPO+PS shows superior performance compared to GRPO+OS, indicating the benefits of using fine-grained, step-aware gradient coefficients. Furthermore, we explore the iterative RL, in our experiments, we conduct two rounds of iteration. As shown in Figure 6, we notice that the iterative RL significantly improves the performance, especially at the first iteration.

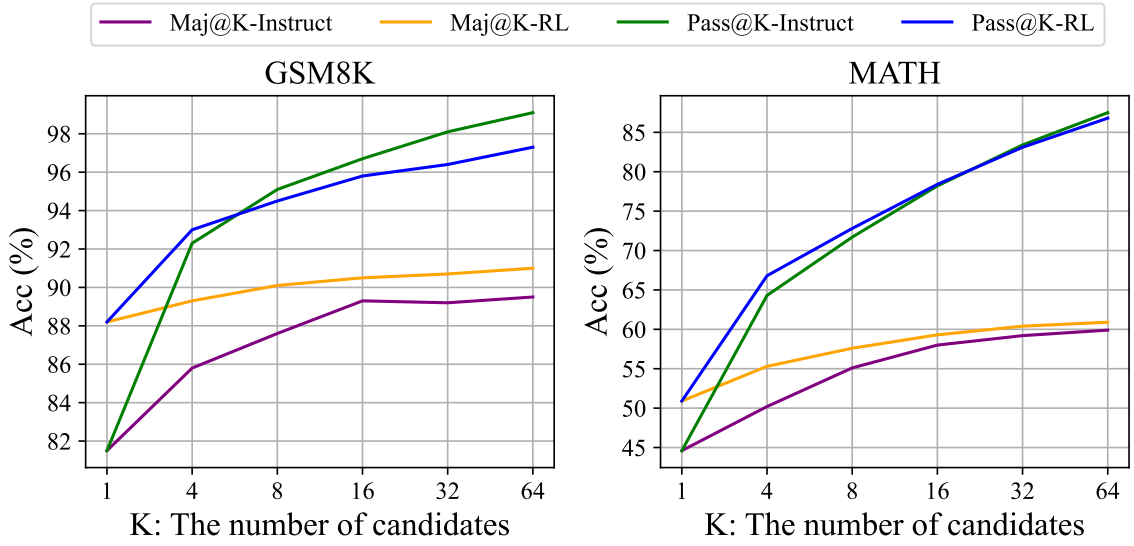


Figure 7 | The Maj@K and Pass@K of SFT and RL DeepSeekMath 7B on GSM8K and MATH (temperature 0.7). It was noted that RL enhances Maj@K but not Pass@K.

5.2.2. *Why RL Works?*

In this paper, we conduct reinforcement learning based on a subset of instruction tuning data, and it achieves significant performance enhancement upon the instruction tuning model. To further explain why reinforcement learning works. We evaluate the Pass@K and Maj@K accuracy of the Instruct and RL models on two benchmarks. As shown in Figure 7, RL enhances Maj@K's performance but not Pass@K. These findings indicate that RL enhances the model's overall performance by rendering the output distribution more robust, in other words, **it seems that the improvement is attributed to boosting the correct response from TopK rather than the enhancement of fundamental capabilities.** Similarly, (Wang et al., 2023a) identified a **misalignment problem** in reasoning tasks within the SFT model, showing that the reasoning performance of SFT models can be improved through a series of preference alignment strategies (Song et al., 2023; Wang et al., 2023a; Yuan et al., 2023b).

5.2.3. *How to Achieve More Effective RL?*

We demonstrate RL works pretty well in mathematical reasoning tasks. We also provide a unified paradigm to understand different representative training methods. Within this paradigm, all methods are conceptualized as either direct or simplified RL techniques. As summarized in Equation 5, there exist three key components: Data Source, Algorithm, and Reward Function. We provide some potential future directions about the three components.

Data Source Data source is the raw material of all training methods. In the context of RL, we specifically refer to the data source as the unlabeled questions with the outputs sampled from the policy model. In this paper, we only use the questions from the instruction tuning stage and a naive nucleus sampling to sample outputs. We think this is a potential reason that our RL pipeline only improves the Maj@K performance. In the future, we will explore our RL pipeline on out-of-distribution question prompts, in conjunction with **advanced sampling (decoding) strategies**, like those based on tree-search methods (Yao et al., 2023). Also, the **efficient inference techniques** (Kwon et al., 2023; Leviathan et al., 2023; Xia et al., 2023, 2024), which determines

the exploration efficiency of policy models, also play an exceedingly important role.

Algorithms Algorithms process the data and reward signal to the gradient coefficient to update the model parameter. Based on Equation 5, to some extent, all methods now fully **TRUST** the signal of the reward function to increase or decrease the conditional probability of a certain token. However, it is impossible to ensure the reward signal is always reliable, especially in extremely complex tasks. For example, even the PRM800K datasets (Lightman et al., 2023), which have been carefully annotated by well-trained annotators, still contain approximately 20% of incorrectly annotations[7]. To this end, we will explore the reinforcement learning algorithm that is robust against noisy reward signals. We believe such **WEAK-TO-STRONG** (Burns et al., 2023) alignment methods will bring a fundamental change to the learning algorithms.

Reward Function Reward function is the source of the training signal. In RL, the reward function is usually the neural reward model. We think there exist three important directions for reward models: 1) **How to enhance the generalization ability of the reward model.** The reward model must be effectively generalized to handle out-of-distribution questions and advanced decoding outputs; otherwise, reinforcement learning may merely stabilize the distribution of LLMs rather than improve their fundamental capabilities; 2) **How to reflect the uncertainty of reward model.** The uncertainty could potentially act as a linking bridge between the weak reward model and the weak-to-strong learning algorithms; 3) **How to efficiently build high-quality process reward models** that can provide fine-grained training signals for the reasoning process (Lightman et al., 2023; Wang et al., 2023b).

6. Conclusion, Limitation, and Future Work

We present DeepSeekMath, which outperforms all open-source models on the competition-level MATH benchmark and approaches the performance of closed models. DeepSeekMath is initialized with DeepSeek-Coder-v1.5 7B and undergoes continual training for 500B tokens, with a significant component of the training data being 120B math tokens sourced from Common Crawl. Our extensive ablation study shows web pages offer significant potential for high-quality mathematical data, while arXiv may not as beneficial as we expected. We introduce Group Relative Policy Optimization (GRPO), a variant of Proximal Policy Optimization (PPO), which can notably improve mathematical reasoning capabilities with less memory consumption. The experiment results show that GRPO is effective even if DeepSeekMath-Instruct 7B has reached a high score on benchmarks. We also provide a unified paradigm to understand a series of methods and summarize several potential directions for more effective reinforcement learning.

Although DeepSeekMath achieves impressive scores on quantitative reasoning benchmarks, its capability on geometry and theorem-proof are relatively weaker than closed models. For instance, in our dry run, the model cannot handle problems related to triangles and ellipses, which may indicate data selection bias in pre-training and fine-tuning. In addition, restricted by the model scale, DeepSeekMath is worse than GPT-4 on few-shot capability. GPT-4 could improve its performance with few-shot inputs, while DeepSeekMath shows similar performance in zero-shot and few-shot evaluation. In the future, we will further improve our engineered data selection pipeline to construct more high-quality pre-trained corpus. In addition, we will explore the potential directions (Section 5.2.3) for more effective reinforcement learning of LLMs.

[7] https://github.com/openai/prm800k/issues/12#issuecomment-1728491852

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

DeepSeek-AI

research@deepseek.com

arXiv:2501.12948v2 [cs.CL] 4 Jan 2026

Abstract

General reasoning represents a long-standing and formidable challenge in artificial intelligence. Recent breakthroughs, exemplified by large language models (LLMs) (Brown et al., 2020; OpenAI, 2023) and chain-of-thought prompting (Wei et al., 2022b), have achieved considerable success on foundational reasoning tasks. However, this success is heavily contingent upon extensive human-annotated demonstrations, and models' capabilities are still insufficient for more complex problems. Here we show that the reasoning abilities of LLMs can be incentivized through pure reinforcement learning (RL), obviating the need for human-labeled reasoning trajectories. The proposed RL framework facilitates the emergent development of advanced reasoning patterns, such as self-reflection, verification, and dynamic strategy adaptation. Consequently, the trained model achieves superior performance on verifiable tasks such as mathematics, coding competitions, and STEM fields, surpassing its counterparts trained via conventional supervised learning on human demonstrations. Moreover, the emergent reasoning patterns exhibited by these large-scale models can be systematically harnessed to guide and enhance the reasoning capabilities of smaller models.

1. Introduction

Reasoning capability, the cornerstone of human intelligence, enables complex cognitive tasks ranging from mathematical problem-solving to logical deduction and programming. Recent advances in artificial intelligence have demonstrated that large language models (LLMs) can exhibit emergent behaviors, including reasoning abilities, when scaled to a sufficient size (Kaplan et al., 2020; Wei et al., 2022a). However, achieving such capabilities in pre-training typically demands substantial computational resources. In parallel, a complementary line of research has demonstrated that large language models can be effectively augmented through chain-of-thought (CoT) prompting. This technique, which involves either providing carefully designed few-shot examples or using minimalistic prompts such as "Let's think step by step"(Kojima et al., 2022; Wei et al., 2022b), enables models to produce intermediate reasoning steps, thereby substantially enhancing their performance on complex tasks. Similarly, further performance gains have been observed when models learn high-quality, multi-step reasoning trajectories during the post-training phase (Chung et al., 2024; OpenAI, 2023). Despite their effectiveness, these approaches exhibit notable limitations. Their dependence on human-annotated reasoning traces hinders scalability and introduces cognitive biases. Furthermore, by constraining models to replicate human thought processes, their performance is inherently capped by the human-

provided exemplars, which prevents the exploration of superior, non-human-like reasoning pathways.

To tackle these issues, we aim to explore the potential of LLMs for developing reasoning abilities through self-evolution in an RL framework, with minimal reliance on human labeling efforts. Specifically, we build upon DeepSeek-V3-Base (DeepSeek-AI, 2024b) and employ Group Relative Policy Optimization (GRPO) (Shao et al., 2024) as our RL framework. The reward signal is solely based on the correctness of final predictions against ground-truth answers, without imposing constraints on the reasoning process itself. Notably, we bypass the conventional supervised fine-tuning (SFT) phase before RL training. This design choice stems from our hypothesis that human-defined reasoning patterns may limit model exploration, whereas unrestricted RL training can better incentivize the emergence of novel reasoning capabilities in LLMs. Through this process, detailed in Section 2, our model (referred to as DeepSeek-R1-Zero) naturally developed diverse and sophisticated reasoning behaviors. In solving reasoning problems, the model exhibits a tendency to generate longer responses, incorporating verification, reflection, and the exploration of alternative approaches within each response. Although we do not explicitly teach the model how to reason, it successfully learns improved reasoning strategies through reinforcement learning.

Although DeepSeek-R1-Zero demonstrates excellent reasoning capabilities, it faces challenges such as poor readability and language mixing, occasionally combining English and Chinese within a single chain-of-thought response. Furthermore, the rule-based RL training stage of DeepSeek-R1-Zero is narrowly focused on reasoning tasks, resulting in limited performance in broader areas such as writing and open-domain question answering. To address these challenges, we introduce DeepSeek-R1, a model trained through a multi-stage learning framework that integrates rejection sampling, reinforcement learning, and supervised fine-tuning, detailed in Section 3. This training pipeline enables DeepSeek-R1 to inherit the reasoning capabilities of its predecessor, DeepSeek-R1-Zero, while aligning model behavior with human preferences through additional non-reasoning data.

To enable broader access to powerful AI at a lower energy cost, we have distilled several smaller models and made them publicly available. These distilled models exhibit strong reasoning capabilities, surpassing the performance of their original instruction-tuned counterparts. We believe that these instruction-tuned versions will also significantly contribute to the research community by providing a valuable resource for understanding the mechanisms underlying long chain-of-thought (CoT) reasoning models and for fostering the development of more powerful reasoning models. We release DeepSeek-R1 series models to the public at `https://huggingface.co/deepseek-ai`.

2. DeepSeek-R1-Zero

We begin by elaborating on the training of DeepSeek-R1-Zero, which relies exclusively on reinforcement learning without supervised fine-tuning. To facilitate large-scale RL efficiency, we adopt Group Relative Policy Optimization (GRPO) (Shao et al., 2024).

2.1. Group Relative Policy Optimization

GRPO (Shao et al., 2024) is the reinforcement learning algorithm that we adopt to train DeepSeek-R1-Zero and DeepSeek-R1. It was originally proposed to simplify the training process and reduce the resource consumption of Proximal Policy Optimization (PPO) (Schulman et al., 2017), which is widely used in the RL stage of LLMs (Ouyang et al., 2022).

For each question q, GRPO samples a group of outputs $\{o_1, o_2, \cdots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_θ by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)]$$
$$\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}A_i, \text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right) - \beta\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right)\right), \tag{1}$$

$$\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right) = \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - \log\frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - 1, \tag{2}$$

where π_{ref} is a reference policy, ε and β are hyper-parameters, and A_i is the advantage, computed using a group of rewards $\{r_1, r_2, \ldots, r_G\}$ corresponding to the outputs within each group:

$$A_i = \frac{r_i - mean(\{r_1, r_2, \cdots, r_G\})}{std(\{r_1, r_2, \cdots, r_G\})}. \tag{3}$$

We give a comparison of GRPO and PPO in Supplementary A.3. To train DeepSeek-R1-Zero, we set the learning rate to 3e-6, the KL coefficient to 0.001, and the sampling temperature to 1 for rollout. For each question, we sample 16 outputs with a maximum length of 32,768 tokens before the 8.2k step and 65,536 tokens afterward. As a result, both the performance and response length of DeepSeek-R1-Zero exhibit a significant jump at the 8.2k step, with training continuing for a total of 10,400 steps, corresponding to 1.6 training epochs. Each training step consists of 32 unique questions, resulting in a training batch size of 512. Every 400 steps, we replace the reference model with the latest policy model. To accelerate training, each rollout generates 8,192 outputs, which are randomly split into 16 mini-batches and trained for only a single inner epoch.

Table 1 | Template for DeepSeek-R1-Zero. prompt will be replaced with the specific reasoning question during training.

A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. The reasoning process and answer are enclosed within `<think>...</think>` and `<answer>...</answer>` tags, respectively, i.e., `<think>` reasoning process here `</think>` `<answer>` answer here `</answer>`. User: prompt. Assistant:

Our high-performance RL infrastructure is described in Supplementary B.1, ensuring scalable and efficient training.

2.2. Reward Design

The reward is the source of the training signal, which decides the direction of RL optimization. For DeepSeek-R1-Zero, we employ rule-based rewards to deliver precise feedback for data in mathematical, coding, and logical reasoning domains. Our rule-based reward system mainly consists of two types of rewards: accuracy rewards and format rewards.

Accuracy rewards evaluate whether the response is correct. For example, in the case of math problems with deterministic results, the model is required to provide the final answer in a specified format (e.g., within a box), enabling reliable rule-based verification of correctness. Similarly, for code competition prompts, a compiler can be utilized to evaluate the model's

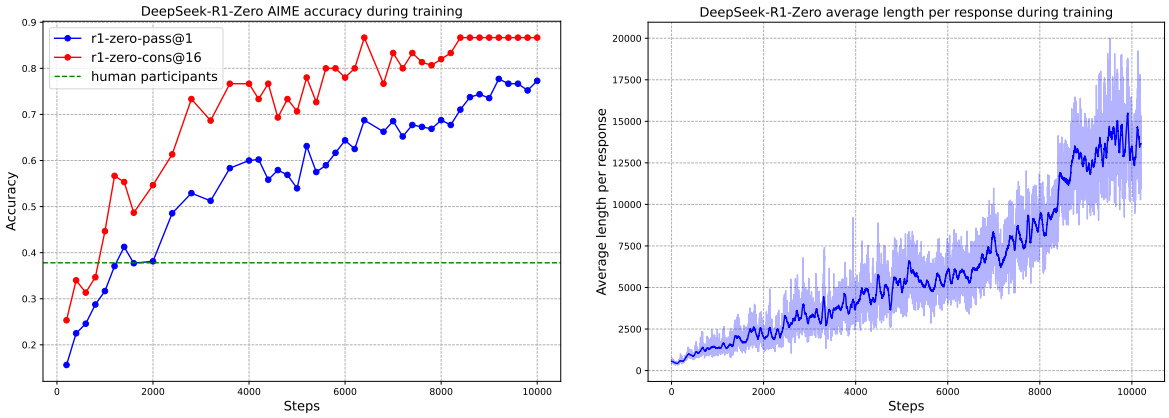


Figure 1 | (a) AIME accuracy of DeepSeek-R1-Zero during training. AIME takes a mathematical problem as input and a number as output, illustrated in Table 32. Pass@1 and Cons@16 are described in Supplementary D.1. The baseline is the average score achieved by human participants in the AIME competition. (b) The average response length of DeepSeek-R1-Zero on the training set during the RL process. DeepSeek-R1-Zero naturally learns to solve reasoning tasks with more thinking time. Note that a training step refers to a single policy update operation.

responses against a suite of predefined test cases, thereby generating objective feedback on correctness.

Format rewards complement the accuracy reward model by enforcing specific formatting requirements. In particular, the model is incentivized to encapsulate its reasoning process within designated tags, specifically '`<think>`' and '`</think>`'. This ensures that the model's thought process is explicitly delineated, enhancing interpretability and facilitating subsequent analysis.

$$Reward_{\text{rule}} = Reward_{\text{acc}} + Reward_{\text{format}} \tag{4}$$

The accuracy, reward and format reward are combined with the same weight. Notably, we abstain from applying neural reward models—whether outcome-based or process-based—to reasoning tasks. This decision is predicated on our observation that neural reward models are susceptible to reward hacking during large-scale reinforcement learning. Moreover, retraining such models necessitates substantial computational resources and introduces additional complexity into the training pipeline, thereby complicating the overall optimization process.

2.3. Incentivize Reasoning Capability in LLMs

Specifically, we apply the RL technique on the DeepSeek-V3 base to train DeepSeek-R1-Zero. During training, we design a straightforward template, to require DeepSeek-R1-Zero to first produce a reasoning process, followed by the final answer. We intentionally limit our constraints to this structural format, avoiding any content-specific biases to ensure that we can accurately observe the model's natural progression during the RL process.

Figure 1(a) depicts the performance trajectory of DeepSeek-R1-Zero on the AIME 2024 benchmark throughout the RL training process, where the average pass@1 score on AIME 2024 shows a significant increase, jumping from an initial 15.6% to 77.9%. In addition, by leveraging the self-consistency decoding (Wang et al., 2023c), the model's performance can be

Table 2 | An interesting "aha moment" of an intermediate version of DeepSeek-R1-Zero. The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to
Response: <think> To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both $\cdots$ $\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2$. Rearrange to isolate the inner square root term: $(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$ $\ldots$ Wait, wait. Wait. That's an aha moment I can flag here. Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$ We started with the equation: $\sqrt{a - \sqrt{a + x}} = x$ First, let's square both sides: $a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$ Next, I could square both sides again, treating the equation: $\cdots$ $\ldots$

further improved, achieving an accuracy of 86.7%. This performance significantly surpasses the average performance across all human competitors. Besides the math competitions, as shown in Figure 10, DeepSeek-R1-Zero also achieves remarkable performance in coding competitions and graduate-level biology, physics, and chemistry problems. These results underscore the effectiveness of RL in enhancing the reasoning capabilities of large language models.

The self-evolution of DeepSeek-R1-Zero exemplifies how RL can autonomously enhance a model's reasoning capabilities.

As shown in Figure 1(b), DeepSeek-R1-Zero exhibits a steady increase in thinking time throughout training, driven solely by intrinsic adaptation rather than external modifications. Leveraging long CoT, the model progressively refines its reasoning, generating hundreds to thousands of tokens to explore and improve its problem-solving strategies.

The increase in thinking time fosters the autonomous development of sophisticated behaviors. Specifically, DeepSeek-R1-Zero increasingly exhibits advanced reasoning strategies such as reflective reasoning and systematic exploration of alternative solutions (see Figure 9(a) in Supplementary C.2 for details), significantly boosting its performance on verifiable tasks like math and coding. Notably, during training, DeepSeek-R1-Zero exhibits an "aha moment" (Table 2), characterized by a sudden increase in the use of the word "wait" during reflections (see Figure 9(b) in Supplementary C.2 for details). This moment marks a distinct change in reasoning patterns and clearly shows the self-evolution process of DeepSeek-R1-Zero.

The self-evolution of DeepSeek-R1-Zero underscores the power and beauty of RL: rather than explicitly teaching the model how to solve a problem, we simply provide it with the right incentives, and it autonomously develops advanced problem-solving strategies. This serves as a reminder of the potential of RL to unlock higher levels of capabilities in LLMs, paving the way for more autonomous and adaptive models in the future.

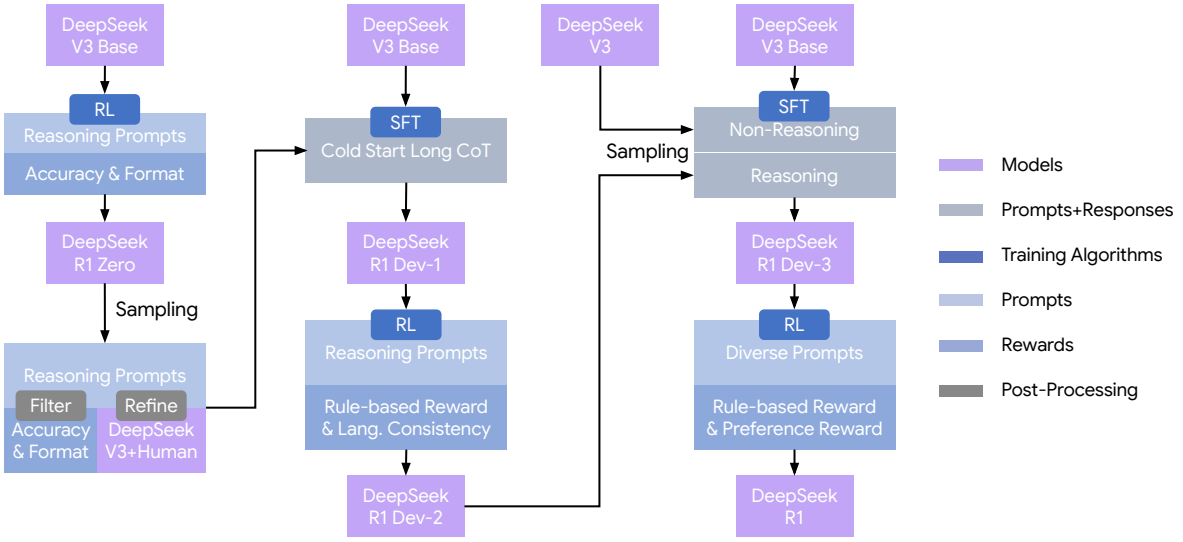


Figure 2 | The multi-stage pipeline of DeepSeek-R1. A detailed background on DeepSeek-V3 Base and DeepSeek-V3 is provided in Supplementary A.1. The models DeepSeek-R1 Dev1, Dev2, and Dev3 represent intermediate checkpoints within this pipeline.

3. DeepSeek-R1

Although DeepSeek-R1-Zero exhibits strong reasoning capabilities, it faces several issues. DeepSeek-R1-Zero struggles with challenges like poor readability, and language mixing, as DeepSeek-V3-Base is trained on multiple languages, especially English and Chinese. To address these issues, we develop DeepSeek-R1, whose pipeline is illustrated in Figure 2.

In the initial stage, we collect thousands of cold-start data that exhibits a conversational, human-aligned thinking process. RL training is then applied to improve the model performance with the conversational thinking process and language consistency. Subsequently, we apply rejection sampling and SFT once more. This stage incorporates both reasoning and non-reasoning datasets into the SFT process, enabling the model to not only excel in reasoning tasks but also demonstrate advanced writing capabilities. To further align the model with human preferences, we implement a secondary RL stage designed to enhance the model's helpfulness and harmlessness while simultaneously refining its reasoning capabilities.

The remainder of this section details the key components of this pipeline: Section 3.1 introduces the Reward Model utilized in our RL stages, and Section 3.2 elaborates on the specific training methodologies and implementation details. Data we used in this stage is detailed in Supplementary B.3.

3.1. Model-based Rewards

For general data, we resort to reward models to capture human preferences in complex and nuanced scenarios. We build upon the DeepSeek-V3 pipeline and adopt a similar distribution of preference pairs and training prompts. For helpfulness, we focus exclusively on the final summary, ensuring that the assessment emphasizes the utility and relevance of the response to the user while minimizing interference with the underlying reasoning process. For harmlessness, we evaluate the entire response of the model, including both the reasoning process and the summary, to identify and mitigate any potential risks, biases, or harmful content that may arise

during the generation process.

Helpful Reward Model Regarding helpful reward model training, we first generate preference pairs by prompting DeepSeek-V3 using the arena-hard prompt format, listed in Supplementary B.2, where each pair consists of a user query along with two candidate responses. For each preference pair, we query DeepSeek-V3 four times, randomly assigning the responses as either Response A or Response B to mitigate positional bias. The final preference score is determined by averaging the four independent judgments, retaining only those pairs where the score difference (Δ) exceeds 1 to ensure meaningful distinctions. Additionally, to minimize length-related biases, we ensure that the chosen and rejected responses of the whole dataset have comparable lengths. In total, we curated 66,000 data pairs for training the reward model. The prompts used in this dataset are all non-reasoning questions and are sourced either from publicly available open-source datasets or from users who have explicitly consented to share their data for the purpose of model improvement. The architecture of our reward model is consistent with that of DeepSeek-R1, with the addition of a reward head designed to predict scalar preference scores.

$$Reward_{helpful} = RM_{helpful}(Response_A, Response_B) \tag{5}$$

The helpful reward models were trained with a batch size of 256, a learning rate of 6e-6, and for a single epoch over the training dataset. The maximum sequence length during training is set to 8192 tokens, whereas no explicit limit is imposed during reward model inference.

Safety Reward Model To assess and improve model safety, we curated a dataset of 106,000 prompts with model-generated responses annotated as “safe" or “unsafe" according to predefined safety guidelines. Unlike the pairwise loss employed in the helpfulness reward model, the safety reward model was trained using a point-wise methodology to distinguish between safe and unsafe responses. The training hyperparameters are the same as the helpful reward model.

$$Reward_{safety} = RM_{safety}(Response) \tag{6}$$

For general queries, each instance is categorized as belonging to either the safety dataset or the helpfulness dataset. The general reward, $Reward_{General}$, assigned to each query corresponds to the respective reward defined within the associated dataset.

3.2. Training Details

3.2.1. Training Details of the First RL Stage

In the first stage of RL, we set the learning rate to 3e-6, the KL coefficient to 0.001, the GRPO clip ratio ε to 10, and the sampling temperature to 1 for rollout. For each question, we sample 16 outputs with a maximum length of 32,768. Each training step consists of 32 unique questions, resulting in a training batch size of 512 per step. Every 400 steps, we replace the reference model with the latest policy model. To accelerate training, each rollout generates 8,192 outputs, which are randomly split into 16 minibatches and trained for only a single inner epoch. However, to mitigate the issue of language mixing, we introduce a language consistency reward during RL training, which is calculated as the proportion of target language words in the CoT.

$$Reward_{language} = \frac{Num(Words_{target})}{Num(Words)} \tag{7}$$

Although ablation experiments in Supplementary B.6 show that such alignment results in a slight degradation in the model's performance, this reward aligns with human preferences, making it more readable. We apply the language consistency reward to both reasoning and non-reasoning data by directly adding it to the final reward.

Note that the clip ratio plays a crucial role in training. A lower value can lead to the truncation of gradients for a significant number of tokens, thereby degrading the model's performance, while a higher value may cause instability during training.

3.2.2. Training Details of the Second RL Stage

Specifically, we train the model using a combination of reward signals and diverse prompt distributions. For reasoning data, we follow the methodology outlined in DeepSeek-R1-Zero, which employs rule-based rewards to guide learning in mathematical, coding, and logical reasoning domains. During the training process, we observe that CoT often exhibits language mixing, particularly when RL prompts involve multiple languages. For general data, we utilize reward models to guide training. Ultimately, the integration of reward signals with diverse data distributions enables us to develop a model that not only excels in reasoning but also prioritizes helpfulness and harmlessness. Given a batch of data, the reward can be formulated as

$$Reward = Reward_{\text{reasoning}} + Reward_{\text{general}} + Reward_{\text{language}} \tag{8}$$

$$\text{where, } Reward_{\text{reasoning}} = Reward_{\text{rule}} \tag{9}$$

$$Reward_{\text{general}} = Reward_{\text{reward_model}} + Reward_{\text{format}} \tag{10}$$

The second stage of RL retains most of the parameters from the first stage, with the key difference being a reduced temperature of 0.7, as we find that higher temperatures in this stage lead to incoherent generation. The stage comprises a total of 1,700 training steps, during which general instruction data and preference-based rewards are incorporated exclusively in the final 400 steps. We find that more training steps with the model based preference reward signal may lead to reward hacking, which is documented in Supplementary B.5. The total training cost is listed in Supplementary B.4.4.

4. Experiment

We evaluate our models on MMLU (Hendrycks et al., 2021), MMLU-Redux (Gema et al., 2025), MMLU-Pro (Wang et al., 2024), C-Eval (Huang et al., 2023), and CMMLU (Li et al., 2024), IFEval (Zhou et al., 2023b), FRAMES (Krishna et al., 2024), GPQA Diamond (Rein et al., 2023), SimpleQA (OpenAI, 2024a), C-SimpleQA (He et al., 2024), SWE-Bench Verified (OpenAI, 2024b), Aider (Gauthier, 2025), LiveCodeBench (Jain et al., 2024) (2024-08 – 2025-01), Codeforces (Mirzayanov, 2025), Chinese National High School Mathematics Olympiad (CNMO 2024) (CMS, 2024), and American Invitational Mathematics Examination 2024 (AIME 2024) (MAA, 2024). The details of these benchmarks are listed in Supplementary D.

Table 3 summarizes the performance of DeepSeek-R1 across multiple developmental stages, as outlined in Figure 2. A comparison between DeepSeek-R1-Zero and DeepSeek-R1 Dev1 reveals substantial improvements in instruction-following, as evidenced by higher scores on the IF-Eval and ArenaHard benchmarks. However, due to the limited size of the cold-start dataset, Dev1 exhibits a partial degradation in reasoning performance compared to DeepSeek-R1-Zero, most notably on the AIME benchmark. In contrast, DeepSeek-R1 Dev2 demonstrates

Cognitive Behaviors that Enable Self-Improving Reasoners, *or,* Four Habits of Highly Effective STaRs

Kanishk Gandhi
Stanford University

Ayush Chakravarthy
Stanford University

Anikait Singh
Stanford University

Nathan Lile
SynthLabs

Noah D. Goodman
Stanford University

arXiv:2503.01307v2 [cs.CL] 15 Aug 2025

Abstract

Test-time inference has emerged as a powerful paradigm for enabling language models to "think" longer and more carefully about complex challenges, much like skilled human experts. While reinforcement learning (RL) can drive self-improvement in language models on verifiable tasks, some models exhibit substantial gains while others quickly plateau. For instance, we find that Qwen-2.5-3B far exceeds Llama-3.2-3B under identical RL training for the game of Countdown. This discrepancy raises a critical question: what intrinsic properties enable effective self-improvement? We introduce a framework to investigate this question by analyzing four key *cognitive behaviors* — verification, backtracking, subgoal setting, and backward chaining — that both expert human problem solvers and successful language models employ. Our study reveals that Qwen naturally exhibits these reasoning behaviors, whereas Llama initially lacks them. In systematic experimentation with controlled behavioral datasets, we find that priming Llama with examples containing these reasoning behaviors enables substantial improvements during RL, matching or exceeding Qwen's performance. Importantly, the presence of reasoning behaviors, rather than correctness of answers, proves to be the critical factor — models primed with incorrect solutions containing proper reasoning patterns achieve comparable performance to those trained on correct solutions. Finally, leveraging continued pretraining with OpenWebMath data, filtered to amplify reasoning behaviors, enables the Llama model to match Qwen's self-improvement trajectory. Our findings establish a fundamental relationship between initial reasoning behaviors and the capacity for improvement, explaining why some language models effectively utilize additional computation while others plateau.[1]

"The limits of my language mean the limits of my world."
—Wittgenstein

1 Introduction

When humans encounter a difficult but solvable problem, we spend more time thinking deeply and deliberately to arrive at a solution. Remarkably, recent language models have begun demonstrating similar reasoning behaviors when trained to self-improve via reinforcement learning (Guo et al., 2025; Jaech et al., 2024). Training language models with reinforcement learning (RL) on verifiable problems is not a new approach (Zelikman et al., 2022; Havrilla et al., 2024; Hoffman et al., 2023), but older methods leveraging RL plateaued after a few iterations without exploring many effective ways to use test-time compute for thinking. In this work, we investigate the reasons behind this change in self-improvement capabilities, focusing on the presence of key cognitive behaviors in base language models.

We focus our investigation on two base models, Qwen-2.5-3B (Qwen et al., 2025) and Llama-3.2-3B (Grattafiori et al., 2024), which show striking differences when trained with

[1]Code available at https://github.com/kanishkg/cognitive-behaviors

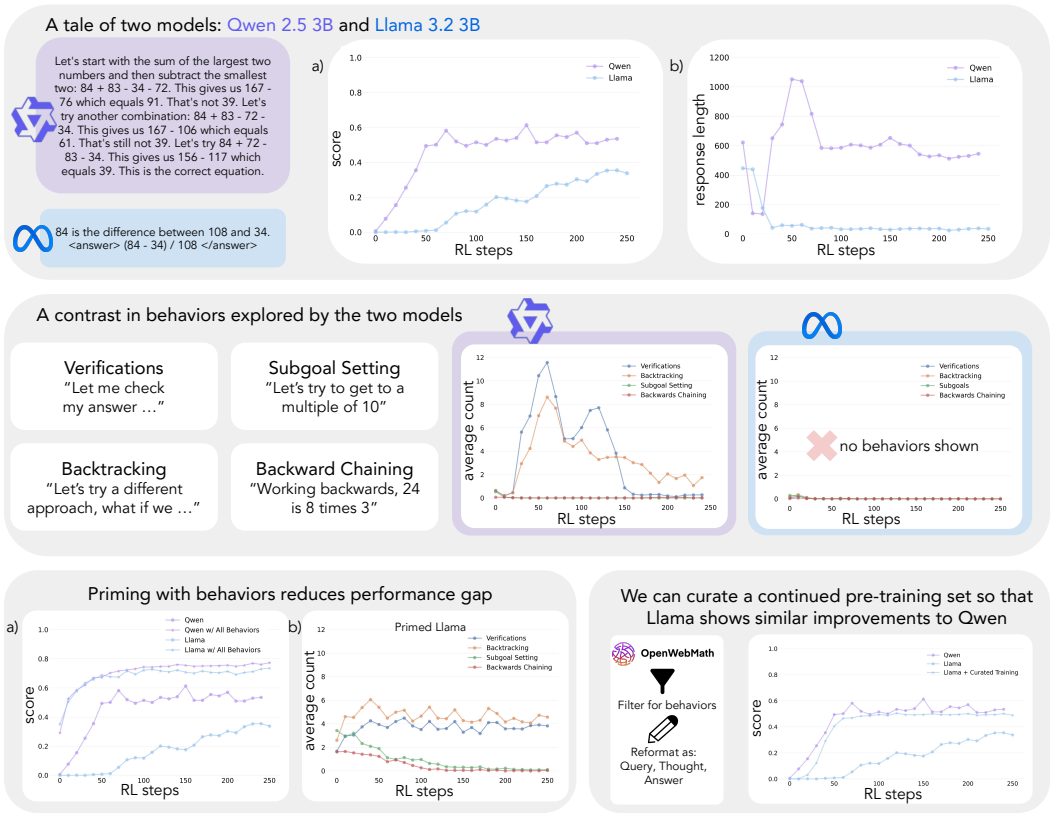


Figure 1: **Comparative analysis of Qwen-2.5-3B and Llama-3.2-3B models with RL on Countdown.** (Top) (a) Performance scores on Countdown for both models (b) Evolution of response lengths throughout RL training. (Middle) Emergence of specific reasoning characteristics as a function of training steps for Qwen-2.5-3B (left) and Llama-3.2-3B (right). (Bottom-Left) (a) Countdown performance when base models are primed with a synthetic dataset of desired reasoning behaviors; (b) Differential impact of RL on reasoning behaviors of the primed Llama3.2-3B: amplification of backtracking and verification contrasted with suppression of backward chaining and subgoal setting. (Bottom-Right) Comparative efficacy of teaching reasoning behaviors through fine-tuning on a curated OpenWebMath dataset, demonstrating that Llama's reasoning capabilities can be improved to match Qwen's through targeted training.

reinforcement learning on the game of Countdown (Countdown, 2024; Gandhi et al., 2024). While Qwen demonstrates substantial improvements in problem-solving ability, Llama[2] shows limited gains under an identical training process. What properties of the initial language model enable such improvements?

To systematically investigate this question, we develop a framework for analyzing *cognitive behaviors* that are useful for solving problems. We characterize four key cognitive behaviors: **verification** (systematic error-checking), **backtracking** (abandoning failing approaches), **subgoal setting** (decomposing problems into manageable steps), and **backward chaining** (reasoning from desired outcomes to initial inputs). These behaviors mirror how expert problem solvers approach difficult tasks — a mathematician verifies each step of a proof, backtracks when encountering contradictions, and breaks complex theorems into simpler lemmas. We examine these four behaviors because they can be used to represent search-based reasoning that goes beyond the typical "linear" reasoning shown by language models.

[2]For the remainder of this paper, we refer to Qwen2.5-3B as "Qwen" and Llama3.2-3B as "Llama", though full model names are also occasionally used. All other models are specified with complete designations.

While many other cognitive behaviors exist, we begin with these as they are well-defined and easily identifiable in model outputs.

Our initial analysis reveals that Qwen naturally exhibits these reasoning behaviors, particularly verification and backtracking, while Llama lacks them. This observation motivates our central hypothesis: certain reasoning behaviors in the initial policy are necessary for efficiently utilizing increased test-time compute through extended reasoning sequences. We test this hypothesis through interventions on the initial model. First, we demonstrate that priming Llama with synthetic reasoning traces containing these behaviors, especially backtracking, enables substantial improvements during RL, matching Qwen's performance trajectory. Second, these gains persist even when primed on incorrect solutions, if they exhibit proper reasoning patterns, suggesting that the presence of reasoning behaviors, rather than access to correct solutions, is the critical factor enabling successful self-improvement. Third, by curating pretraining data from OpenWebMath (Paster et al., 2023) that emphasizes these reasoning behaviors, we show that targeted modification of the pretraining distribution can successfully induce the behavioral patterns necessary for efficient use of test-time compute — the improvement trajectory of Llama matches that of Qwen.

Our investigation reveals a strong relationship between a model's initial reasoning behaviors and its capacity for improvement. This connection helps explain why some language models discover effective ways to use additional compute while others plateau. Understanding these dynamics may be key to developing AI systems that can meaningfully improve their problem-solving abilities.

2 Related Work

Recent approaches to improving reasoning capabilities in language models can be broadly categorized into three complementary directions: external search methods that leverage multiple samples, in-context search that enables models to reason over their own outputs, and reinforcement learning approaches that allow models to discover reasoning strategies autonomously.

External Search for Reasoning. Recent work has shown that language models can significantly improve their performance on complex tasks when given additional inference-time compute. Snell et al. (2024) systematically explore this space by developing various methods to search through reasoning trajectories. These approaches range from simple parallel sampling (Li et al., 2024; Brown et al., 2024) to more sophisticated methods using verifiers or process reward models (PRMs) (Lightman et al., 2023; Yao et al., 2023). Some researchers have taken this further by using the search process itself to improve the underlying reasoning model (Wang et al., 2023; Luo et al., 2024). However, these methods typically operate without awareness of previously explored solutions, limiting their efficiency through redundant exploration.

In-Context Search and Self-Improvement. In contrast to external search methods, another line of research focuses on enabling models to search sequentially in language. This has been achieved through various approaches such as 1) in-context examples (Gandhi et al., 2023), 2) finetuning on linearized search traces (Gandhi et al., 2024; Lehnert et al., 2024), and 3) training on self-correction examples (Ye et al., 2024; Qu et al., 2025; Kumar et al., 2024; Hwang et al., 2024). Recent work by Schultz et al. (2024) bridges the gap between in-context and external search methods, demonstrating improved performance on strategic games (cf. Xiang et al., 2025). While effective, these approaches often require careful engineering of training data to incorporate desired behaviors like self-correction and backtracking.

Reinforcement Learning for Reasoning. The prospect of models autonomously discovering effective reasoning strategies has motivated significant research in reinforcement learning approaches. Early work in teaching language models to reason with verifiable outcomes explored various RL methodologies, from off-policy and batch methods (Zelikman et al., 2022; Havrilla et al., 2024; Hoffman et al., 2023) to on-policy approaches (Zelikman et al., 2024; Kazemnejad et al., 2024; Cui et al., 2025). These methods differ in their approaches to credit assignment in reasoning trajectories. A notable breakthrough

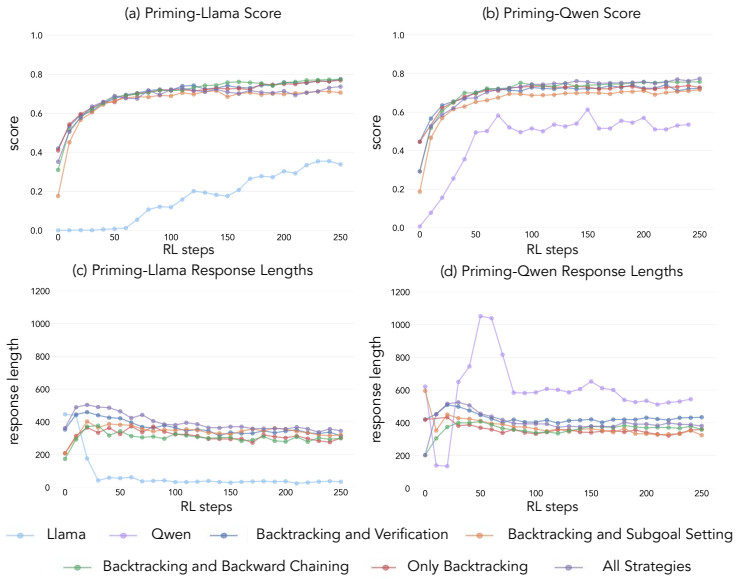


Figure 2: **The effects of priming with different cognitive behaviors.** (a, b) Performance comparison on the Countdown task between Llama-3.2-3B, Qwen-2.5-3B and their primed variants, illustrating the influence of reasoning behavior priming on scores. (c, d) Response length analysis for standard and primed Llama and Qwen models, showing how priming influences reasoning length.

came with Deepseek-R1 (Guo et al., 2025; Jaech et al., 2024), which demonstrated that even a simplified version of PPO (Schulman et al., 2017) called GRPO (Shao et al., 2024) could lead to significant improvements and the emergence of in-context search behaviors. Recent analyses (Li et al., 2025), Wu et al. (2024), Liu et al. (2025), Ye et al. (2025) and Yeo et al. (2025) have begun to unpack these results, revealing that supervised fine-tuning with long, structured chains of thought enhances both the efficiency and performance of RL compared to shorter reasoning chains. However, a crucial question remains unanswered: why do some models successfully learn through RL while others fail to improve? (Yeo et al., 2025)briefly discuss this issue by analyzing the frequency of phrases related to reflective reasoning in the pretraining dataset but their analysis remains limited. Our work addresses this gap by investigating the essential properties of initial models that enable successful reinforcement learning of reasoning behaviors.

3 Identifying and Engineering Self-Improving Behavior

3.1 Initial Investigation: A tale of two models

We begin by investigating a surprising observation: language models of comparable size but from different families show markedly different capacities for improvement through reinforcement learning. The Countdown game serves as our primary testbed — a mathematical puzzle where players must combine a set of input numbers using the four basic arithmetic operations ($+, -, \times, \div$) to reach a target number. For example, given the numbers 25, 30, 3, 4 and a target of 32, the solution involves combining these numbers through a series of operations to reach exactly 32: $(30 - 25 + 3) \times 4$.

We selected Countdown for our analysis because it demands mathematical reasoning, planning, and search strategies that mirror key aspects of general problem-solving similar to other reasoning domains such as math. Unlike more complex domains, Countdown offers a restricted search space that enables tractable analysis while still requiring sophisticated reasoning. Additionally, success in the game depends more on problem-solving abilities than mathematical knowledge, compared to other mathematical tasks where domain knowledge might confound the evaluation of reasoning capabilities.

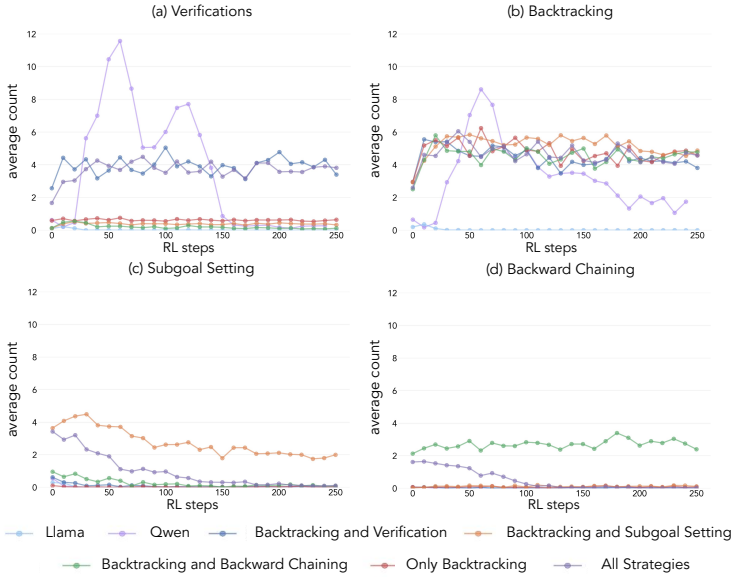


Figure 3: **Analysis of four key reasoning behaviors with Llama-3.2-3B, Qwen-2.5-3B, and primed versions of Llama-3.2-3B.** Plots show mean frequency of (a) solution verification steps, (b) problem-solving backtracking instances, (c) explicit subgoal setting, and (d) backward chaining reasoning approaches across different tasks.

We use two base models to contrast how learning varies across model families: Qwen-2.5-3B and Llama-3.2-3B. Our reinforcement learning experiments build on the VERL library (Sheng et al., 2024), utilizing the TinyZero (Pan et al., 2025) implementation. We train models with PPO (Schulman et al., 2017) for 250 steps, sampling 4 trajectories per prompt. We chose PPO over alternatives like GRPO (Shao et al., 2024) and REINFORCE (Williams, 1992; Hu, 2025; Ahmadian et al., 2024) as it demonstrated superior stability across hyperparameter settings, though performance was anecdotally similar across algorithms.

The results reveal strikingly different learning trajectories. Though both models start at a similar, low performance on the task, Qwen demonstrates a qualitative shift around step 30, characterized by significantly longer responses and improved accuracy (Fig. 1 (top)). By the end of training, Qwen achieves approximately 60% accuracy, substantially outperforming Llama's 30%. Later in training, we observe an interesting change in Qwen's behavior: the model transitions from explicit verification statements in language, "8*35 is 280 which is too high" to implicit solution checking, where the model sequentially tries different solutions until it finds the right answer without using words to evaluate its own work.

This contrast raises a fundamental question: what underlying capabilities enable successful reasoning-based improvement? To answer this, we need a systematic framework for analyzing cognitive behaviors.

3.2 A Framework for Analyzing Cognitive Behaviors

To understand these divergent learning trajectories, we develop a framework for identifying and analyzing key behaviors in model outputs. We focus on four fundamental behaviors: (1) Backtracking or the explicit revision of approaches when errors are detected (e.g., "This approach won't work because..."), (2) Verification or the systematic checking of intermediate results (e.g., "Let's verify this result by..."), (3) Subgoal Setting, where a complex problem is broken down into manageable steps (e.g., "To solve this, we first need to..."), and (4) Backward Chaining, where in a goal-directed reasoning problem, the solution works backwards from a desired outcomes (e.g., "To reach the target of 75, we need a number divisible by...").

We selected these behaviors because they represent problem-solving strategies that deviate from the linear, monotonic reasoning patterns commonly observed in language models. These behaviors enable more dynamic, search-like reasoning trajectories where solutions

can evolve non-linearly. While this set is not exhaustive, these behaviors were chosen because they are easy to identify and they align naturally with human problem-solving strategies in both the Countdown game and broader mathematical reasoning tasks like proof construction.

Each behavior can be identified by its pattern in the reasoning tokens. Backtracking is seen as token sequences that explicitly contradict and replace previous steps, verification produces tokens that compare outcomes against solution criteria, backward chaining generates tokens that construct solution paths from the goal to the initial state, and subgoal setting explicitly proposes an intermediate step to target on the path to the final goal. We develop a classification pipeline using GPT-4o-mini[3] to reliably identify these patterns in the outputs of the model (see App. F for analysis of agreement with humans and larger models).

3.3 The Role of Initial Behaviors in Self-Improvement

Applying this framework to our initial experiment reveals a key insight: Qwen's dramatic performance improvements coincided with the emergence of cognitive behaviors, particularly verification and backtracking (Fig. 1 (middle)). Llama, in contrast, showed minimal evidence of these behaviors throughout training.

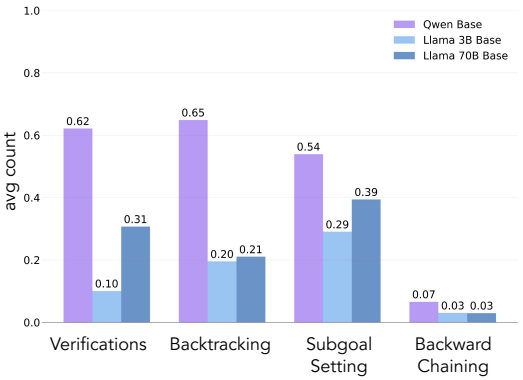


Figure 4: **Exploration of different reasoning behaviors in base models**. An analysis with Qwen2.5-3B, Llama3.2-3B, and Llama3.1-70B on Countdown.

To better understand this disparity, we analyzed the baseline reasoning patterns across three models: Qwen-2.5-3B, Llama-3.2-3B, and Llama-3.1-70B. Our analysis revealed that Qwen-2.5-3B naturally exhibits substantially higher rates of all four behaviors compared to both Llama variants (Fig. 4). While the larger Llama-3.1-70B showed generally increased activation of these behaviors compared to Llama-3.2-3B, this improvement was notably uneven — backtracking, in particular, remained limited even in the larger model. These observations suggest two insights: 1) certain cognitive behaviors in the initial policy may be necessary for models to effectively utilize increased test-time compute through extended reasoning sequences 2) increased model scale can improve the contextual activation of these behaviors. This pattern is particularly significant because reinforcement learning can only amplify behaviors that appear in successful trajectories — making these initial behavioral capabilities a precondition for effective learning.

3.4 Intervening on initial behaviors

Having established the importance of cognitive behaviors in base models, we next investigate whether we could artificially induce these behaviors through targeted interventions. Our hypothesis is that by creating variants of base models that selectively exhibit specific cognitive behaviors before RL training, we can better understand which behavioral patterns are crucial for enabling effective learning.

We begin by curating seven distinct priming datasets using Countdown problems. Five of these datasets emphasize different behavioral combinations: all strategies combined, backtracking only, backtracking with verification, backtracking with subgoal setting, and backtracking with backward chaining. We generate these datasets using Claude-3.5-Sonnet[4], leveraging its ability to produce reasoning trajectories with precisely specified behavioral characteristics. While Claude does not always produce the correct answer (see Fig. 9), it consistently demonstrates the requested reasoning patterns, providing clean behavioral

[3]We chose GPT-4o-mini to balance inference cost and model capability

[4]We chose Claude-3.5-Sonnet as it reliably followed instructions to show the desired behaviors we wanted in the reasoning trajectories.

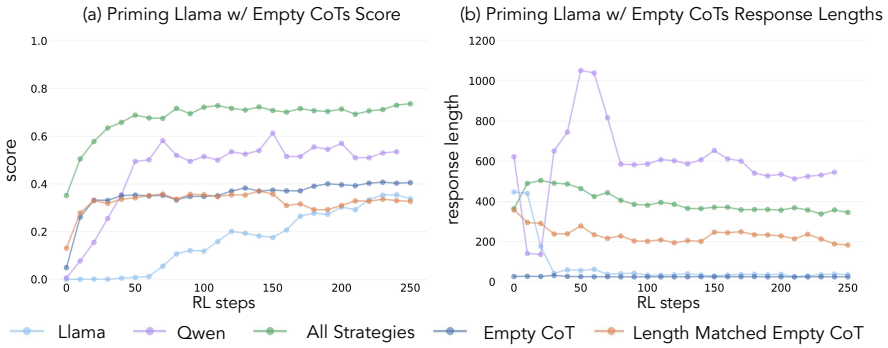


Figure 5: **Impact of Empty Chain-of-Thought Priming on Model Performance and Response Characteristics** Comparative analysis of (a) performance scores on the Countdown task and (b) response length distributions across different model configurations: baseline models (Llama3.2-3B, Qwen2.5-3B), unprimed conditions, length-matched empty CoT priming, and all-strategies-primed Llama3.2-3B.

primitives for our analysis. To verify that improvement stems from specific cognitive behaviors rather than simply increased computation time, we introduce two control conditions: 1) an empty chain-of-thought, where the thoughts are empty, "<*think*></*think*>" and 2) a chain-of-thought where the thoughts have the same length as those of the all-strategies control, but filled with placeholder tokens, "<*think*>. . . . . </*think*>". We also create a variant of our all-strategies dataset containing only incorrect solutions while maintaining the desired reasoning patterns. This variant allows us to disentangle the importance of cognitive behaviors from the accuracy of solutions.

Priming with different behaviors When initialized with datasets containing backtracking behaviors, both Llama and Qwen demonstrate substantial improvements through RL training (Fig. 2). Behavioral analysis reveals that RL selectively amplifies empirically useful behaviors while suppressing others (Fig. 3). For instance, in the all-strategies condition (Fig. 1 (bottom-left)), the models retain and strengthen backtracking and verification while diminishing backward chaining and subgoal setting. However, the suppressed behaviors (backward chaining and subgoal setting) when paired only with backtracking persist through training.

Testing Behavioral Necessity. When primed with the empty chain-of-thought controls, in both conditions, the models performance is comparable to the base Llama model (≈30-35%; see Fig. 5), demonstrating that the mere allocation of additional tokens without the inclusion of cognitive behaviors fails to enable effective use of test-time compute. Further, training with an empty chain-of-thought has a detrimental effect, where the Qwen model stops exploring the behaviors. This suggests that these cognitive behaviors are specifically necessary for models to make productive use of extended computation through longer reasoning sequences.

Behaviors versus Correctness. Surprisingly, models primed with incorrect solutions, but with the right behaviors achieve identical performance to those trained on datasets with correct solutions (Fig. 6). This suggests that the presence of cognitive behaviors in the priming data, rather than the access to correct solutions, is the crucial factor enabling successful self-improvement through reinforcement learning. This extends prior work demonstrating learning from corrupted reasoning trajectories (Li et al., 2025) in a significant way — we show that reasoning patterns from weaker models can effectively bootstrap the learning process to build more capable ones, suggesting that the presence of cognitive behaviors matter more than just the correctness of the outcomes.

The above results show that certain cognitive behaviors are necessary for self-improvement. However, our priming method for inducing behaviors in the initial model was domain-specific, relying on the Countdown game. This may adversely impact the generalization of

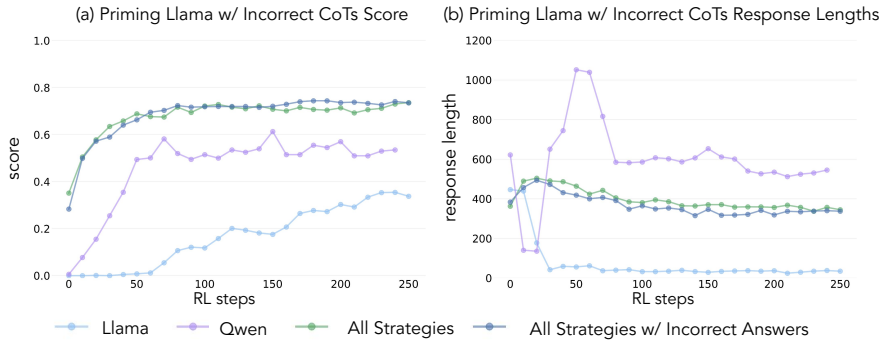


Figure 6: **Effects of Incorrect Chain-of-Thought Priming on Model Performance and Output Characteristics** Evaluation of (a) Countdown task performance scores and (b) response length distributions comparing four conditions: Llama3.2-3B and Qwen2.5-3B base models, Llama3.2-3B with correct all-strategy priming, and Llama3.2-3B primed with incorrect reasoning examples.

the resulting reasoning. Can we instead enable self-improvement by modifying a model's pretraining distribution to increase the frequency of beneficial reasoning behaviors?

Behavioral Frequencies in Pretraining Data. We first analyze the natural frequency of cognitive behaviors in pre-training data, focusing on OpenWebMath (Paster et al., 2023), and FineMath (Allal et al., 2025), which are constructed specifically for mathematical reasoning. Using Qwen-2.5-32B as a classifier[5], we analyze 200,000 randomly sampled documents for the presence of our target behaviors. Even in this math-focused corpus, cognitive behaviors such as backtracking and verifications appear infrequently, suggesting that standard pretraining provides limited exposure to these crucial patterns (see Fig. 7).

3.5 Selectively amplifying behaviors in pretraining data

Behavioral Augmentation of the Pretraining Data. To test whether artificially increasing exposure to cognitive behaviors enhances the potential for self-improvement, we develop a targeted continued pretraining dataset from OpenWebMath. First, using Qwen-2.5-32B as a classifier, we analyze mathematical documents from the pretraining corpus for the presence of our target reasoning behaviors. This allows us to create two contrasting sets: one with the cognitive behaviors and a control set that shows minimal evidence of these behaviors. We then use Qwen-2.5-32B to rewrite each document in the set into a structured question-thought-answer format, preserving the natural presence or absence of cognitive behaviors from the source documents. The final pretraining datasets each contain a total of 8.3 million tokens. This approach allows us to isolate the impact of reasoning behaviors while controlling for the format and amount of mathematical content during pre-training.

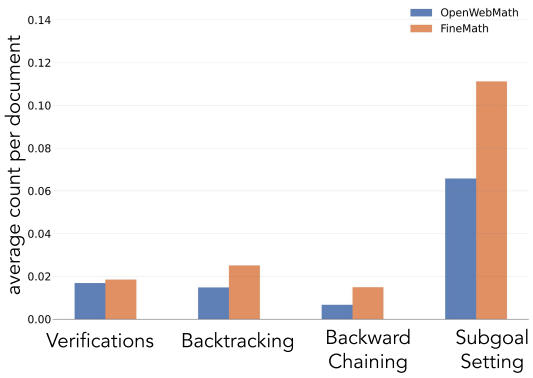


Figure 7: **Behaviors present in Math Pretraining Datasets**. An analysis of the behaviors present in 200,000 randomly sampled documents of OpenWebMath and FineMath. We measure the average count of the behaviors in each document.

[5]We use Qwen-2.5-32B to balance throughput, capability and cost.

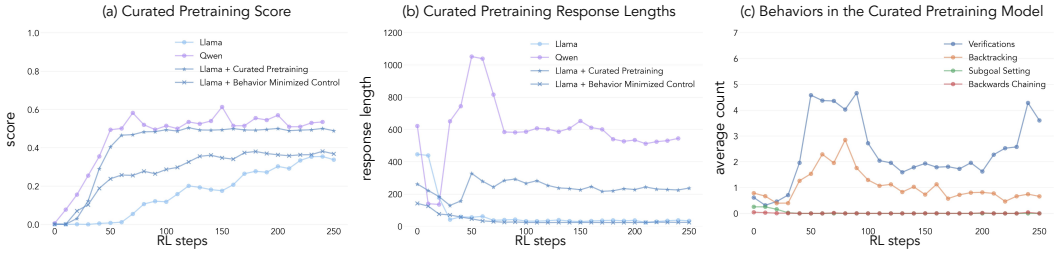


Figure 8: **Impact of curated pretraining on model performance and behavior.** (a) Comparison of performance scores across base models (Llama-3.2-3B, Qwen-2.5-3B) and Llama variants with curated pretraining versus behavior-minimized control. (b) Evolution of response lengths during training for each model configuration. (c) Emergence and development of specific cognitive behaviors in the curated pretraining model over training steps.

After pre-training Llama-3.2-3B on these datasets and applying reinforcement learning, we observe that: 1) the behavior-enriched model achieves performance comparable to Qwen, while the control model shows limited improvement (Fig. 8a) and 2) behavioral analysis of the trained models reveals that the behavior-enriched variant maintains high activation of reasoning behaviors throughout training, while the control shows behaviors similar to the base Llama model (Fig. 8c). These results demonstrate that targeted modification of pre-training data can successfully induce the cognitive behaviors necessary for effective self-improvement through reinforcement learning.

4 Discussion

We have found that a model's initial exploration of cognitive behaviors – particularly its tendency toward verification, backtracking, subgoal setting, and backward chaining – plays a crucial role in enabling self-improvement. Models that naturally exhibit these reasoning behaviors (such as Qwen-2.5-3B) show dramatically better improvement through RL compared to models lacking these behaviors (such as Llama-3.2-3B). Priming models with cognitive behaviors, by a small amount of finetuning, enabled significant performance gains even in models that initially lack these capabilities. Remarkably, this holds even when primed with incorrect solutions that exhibit the target behavioral patterns, suggesting that cognitive behaviors matter more than solution accuracy. Together these results indicate that the presence of cognitive behaviors is a *causal* factor enabling self-improvement through RL. Our initial priming experiments used data based on the Countdown game for training, possibly limiting generality. We thus developed a more diverse behavior-enriched training dataset derived from OpenWebMath. Training Llama on this set then led to self-improvement comparable to Qwen, demonstrating that the capacity for improvement can be engineered through careful curation of pretraining data.

When humans try to solve problems that are difficult but not unsolvable for them, they exhibit certain behaviors that support the problem-solving processes, structuring search over the space of possible solutions to a problem. These *cognitive behaviors* are usually sequential, deliberate and dependent on the problem space (Simon & Newell, 1971). Correspondingly, the cognitive behaviors that are amplified or suppressed during RL training are likely to be highly dependent on the tasks and environments being optimized for. In our studies using Countdown, backtracking and verification were the most critical. This raises important questions about the patterns that enable self-improvement in tasks such as coding, game play or creative writing. We believe that the principle described here will extend to other domains, but future work should explore how task-specific constraints interact with cognitive behaviors. Further, the cognitive behaviors specified in this work are not exhaustive; other behaviors are worth exploring, such as making analogies (Mitchell, 2021) and identifying one's existing state of knowledge (Metcalfe, 1986).

In conclusion, our findings show how cognitive behaviors enable language models to show self-improvement — effectively using increased test-time compute to solve increasingly more challenging problems. Humanity has a rich inheritance of cognitive behaviors that

enable effective reasoning. Future artificial intelligence may go beyond learning to use these existing behaviors – it may discover new ones, potentially revealing entirely new approaches to reasoning and computation.

Acknowledgments

We would like to thank Charlie Snell, Eric Zelikman, Archit Sharma, Rafael Rafailov, Alex Havrilla, Ulyana Piterburg, Chase Blagden, Dimitris Papailiopoulos, and Jan-Philipp Fränken for their discussions and support. KG was supported by an HAI-SAP Grant and an NSF Expeditions grant; AS was supported by the NSF GRFP Fellowship. Compute for this work was provided by Synthlabs, Stanford Marlowe and the Toyota Research Institute (TRI).

References

Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms. *arXiv preprint arXiv:2402.14740*, 2024.

Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, Joshua Lochner, Caleb Fahlgren, Xuan-Son Nguyen, Clémentine Fourrier, Ben Burtenshaw, Hugo Larcher, Haojun Zhao, Cyril Zakka, Mathieu Morlon, Colin Raffel, Leandro von Werra, and Thomas Wolf. Smollm2: When smol goes big – data-centric training of a small language model, 2025. URL https://arxiv.org/abs/2502.02737.

Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling, 2024. URL https://arxiv.org/abs/2407.21787.

Countdown. Countdown (game show). Online, 2024. URL https://en.wikipedia.org/wiki/Countdown_(game_show). [Accessed 29-March-2024].

Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, Qixin Xu, Weize Chen, et al. Process reinforcement through implicit rewards. *arXiv preprint arXiv:2502.01456*, 2025.

Kanishk Gandhi, Dorsa Sadigh, and Noah D Goodman. Strategic reasoning with language models. *arXiv preprint arXiv:2305.19165*, 2023.

Kanishk Gandhi, Denise HJ Lee, Gabriel Grand, Muxin Liu, Winson Cheng, Archit Sharma, and Noah Goodman. Stream of search (sos): Learning to search in language. In *First Conference on Language Modeling*, 2024.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, Danny Wyatt, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Francisco Guzmán, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Govind Thattai, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jack Zhang, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar,

Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

Jason Wei **Xuezhi Wang** **Dale Schuurmans** **Maarten Bosma**

Brian Ichter **Fei Xia** **Ed H. Chi** **Quoc V. Le** **Denny Zhou**

Google Research, Brain Team
{jasonwei,dennyzhou}@google.com

Abstract

We explore how generating a *chain of thought*—a series of intermediate reasoning steps—significantly improves the ability of large language models to perform complex reasoning. In particular, we show how such reasoning abilities emerge naturally in sufficiently large language models via a simple method called *chain-of-thought prompting*, where a few chain of thought demonstrations are provided as exemplars in prompting.

Experiments on three large language models show that chain-of-thought prompting improves performance on a range of arithmetic, commonsense, and symbolic reasoning tasks. The empirical gains can be striking. For instance, prompting a PaLM 540B with just eight chain-of-thought exemplars achieves state-of-the-art accuracy on the GSM8K benchmark of math word problems, surpassing even finetuned GPT-3 with a verifier.

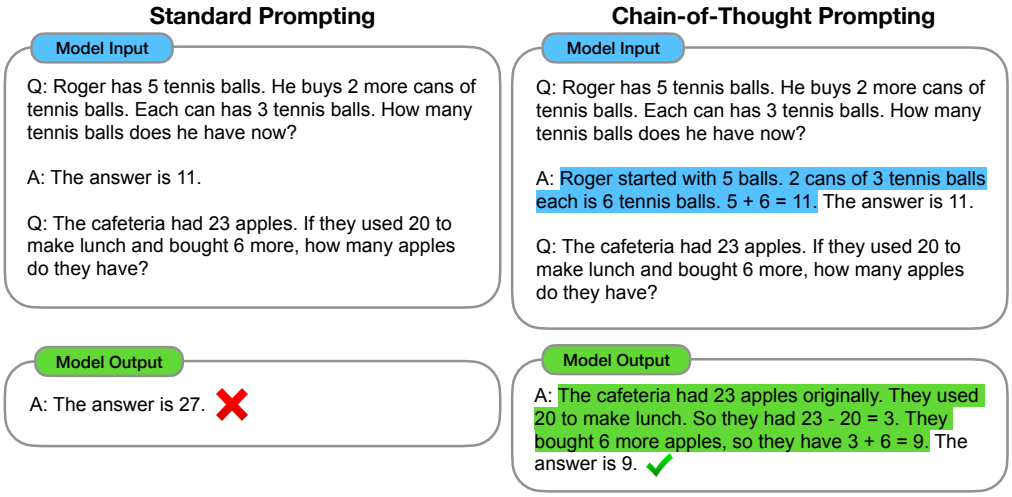


Figure 1: Chain-of-thought prompting enables large language models to tackle complex arithmetic, commonsense, and symbolic reasoning tasks. Chain-of-thought reasoning processes are highlighted.

36th Conference on Neural Information Processing Systems (NeurIPS 2022).

1 Introduction

The NLP landscape has recently been revolutionized by language models (Peters et al., 2018; Devlin et al., 2019; Brown et al., 2020, *inter alia*). Scaling up the size of language models has been shown to confer a range of benefits, such as improved performance and sample efficiency (Kaplan et al., 2020; Brown et al., 2020, *inter alia*). However, scaling up model size alone has not proved sufficient for achieving high performance on challenging tasks such as arithmetic, commonsense, and symbolic reasoning (Rae et al., 2021).

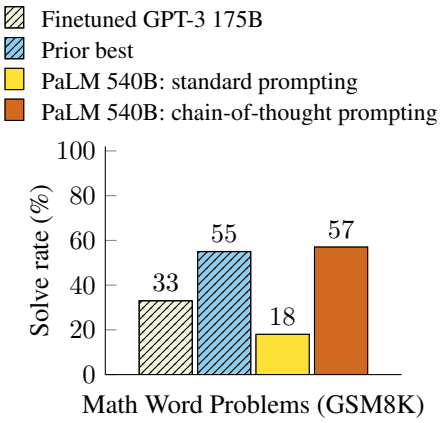


Figure 2: PaLM 540B uses chain-of-thought prompting to achieve new state-of-the-art performance on the GSM8K benchmark of math word problems. Finetuned GPT-3 and prior best are from Cobbe et al. (2021).

This work explores how the reasoning ability of large language models can be unlocked by a simple method motivated by two ideas. First, techniques for arithmetic reasoning can benefit from generating natural language rationales that lead to the final answer. Prior work has given models the ability to generate natural language intermediate steps by training from scratch (Ling et al., 2017) or finetuning a pretrained model (Cobbe et al., 2021), in addition to neuro-symbolic methods that use formal languages instead of natural language (Roy and Roth, 2015; Chiang and Chen, 2019; Amini et al., 2019; Chen et al., 2019). Second, large language models offer the exciting prospect of in-context few-shot learning via *prompting*. That is, instead of finetuning a separate language model checkpoint for each new task, one can simply "prompt" the model with a few input–output exemplars demonstrating the task. Remarkably, this has been successful for a range of simple question-answering tasks (Brown et al., 2020).

Both of the above ideas, however, have key limitations. For rationale-augmented training and finetuning methods, it is costly to create a large set of high quality rationales, which is much more complicated than simple input–output pairs used in normal machine learning. For the traditional few-shot prompting method used in Brown et al. (2020), it works poorly on tasks that require reasoning abilities, and often does not improve substantially with increasing language model scale (Rae et al., 2021). In this paper, we combine the strengths of these two ideas in a way that avoids their limitations. Specifically, we explore the ability of language models to perform few-shot prompting for reasoning tasks, given a prompt that consists of triples: ⟨input, *chain of thought*, output⟩. A *chain of thought* is a series of intermediate natural language reasoning steps that lead to the final output, and we refer to this approach as *chain-of-thought prompting*. An example prompt is shown in Figure 1.

We present empirical evaluations on arithmetic, commonsense, and symbolic reasoning benchmarks, showing that chain-of-thought prompting outperforms standard prompting, sometimes to a striking degree. Figure 2 illustrates one such result—on the GSM8K benchmark of math word problems (Cobbe et al., 2021), chain-of-thought prompting with PaLM 540B outperforms standard prompting by a large margin and achieves new state-of-the-art performance. A prompting only approach is important because it does not require a large training dataset and because a single model checkpoint can perform many tasks without loss of generality. This work underscores how large language models can learn via a few examples with natural language data about the task (c.f. automatically learning the patterns underlying inputs and outputs via a large training dataset).

2 Chain-of-Thought Prompting

Consider one's own thought process when solving a complicated reasoning task such as a multi-step math word problem. It is typical to decompose the problem into intermediate steps and solve each before giving the final answer: *"After Jane gives 2 flowers to her mom she has 10 . . . then after she gives 3 to her dad she will have 7 . . . so the answer is 7."* The goal of this paper is to endow language models with the ability to generate a similar *chain of thought*—a coherent series of intermediate reasoning steps that lead to the final answer for a problem. We will show that sufficiently large

language models can generate chains of thought if demonstrations of chain-of-thought reasoning are provided in the exemplars for few-shot prompting.

Figure 1 shows an example of a model producing a chain of thought to solve a math word problem that it would have otherwise gotten incorrect. The chain of thought in this case resembles a solution and can interpreted as one, but we still opt to call it a chain of thought to better capture the idea that it mimics a step-by-step thought process for arriving at the answer (and also, solutions/explanations typically come *after* the final answer (Narang et al., 2020; Wiegreffe et al., 2022; Lampinen et al., 2022, *inter alia*)).

Chain-of-thought prompting has several attractive properties as an approach for facilitating reasoning in language models.

1. First, chain of thought, in principle, allows models to decompose multi-step problems into intermediate steps, which means that additional computation can be allocated to problems that require more reasoning steps.
2. Second, a chain of thought provides an interpretable window into the behavior of the model, suggesting how it might have arrived at a particular answer and providing opportunities to debug where the reasoning path went wrong (although fully characterizing a model's computations that support an answer remains an open question).
3. Third, chain-of-thought reasoning can be used for tasks such as math word problems, commonsense reasoning, and symbolic manipulation, and is potentially applicable (at least in principle) to any task that humans can solve via language.
4. Finally, chain-of-thought reasoning can be readily elicited in sufficiently large off-the-shelf language models simply by including examples of chain of thought sequences into the exemplars of few-shot prompting.

In empirical experiments, we will observe the utility of chain-of-thought prompting for arithmetic reasoning (Section 3), commonsense reasoning (Section 4), and symbolic reasoning (Section 5).

3 Arithmetic Reasoning

We begin by considering math word problems of the form in Figure 1, which measure the arithmetic reasoning ability of language models. Though simple for humans, arithmetic reasoning is a task where language models often struggle (Hendrycks et al., 2021; Patel et al., 2021, *inter alia*). Strikingly, chain-of-thought prompting when used with the 540B parameter language model performs comparably with task-specific finetuned models on several tasks, even achieving new state of the art on the challenging GSM8K benchmark (Cobbe et al., 2021).

3.1 Experimental Setup

We explore chain-of-thought prompting for various language models on multiple benchmarks.

Benchmarks. We consider the following five math word problem benchmarks: **(1)** the **GSM8K** benchmark of math word problems (Cobbe et al., 2021), **(2)** the **SVAMP** dataset of math word problems with varying structures (Patel et al., 2021), **(3)** the **ASDiv** dataset of diverse math word problems (Miao et al., 2020), **(4)** the **AQuA** dataset of algebraic word problems, and **(5)** the **MAWPS** benchmark (Koncel-Kedziorski et al., 2016). Example problems are given in Appendix Table 12.

Standard prompting. For the baseline, we consider standard few-shot prompting, popularized by Brown et al. (2020), in which a language model is given in-context exemplars of input–output pairs before outputting a prediction for a test-time example. Exemplars are formatted as questions and answers. The model gives the answer directly, as shown in Figure 1 (left).

Chain-of-thought prompting. Our proposed approach is to augment each exemplar in few-shot prompting with a chain of thought for an associated answer, as illustrated in Figure 1 (right). As most of the datasets only have an evaluation split, we manually composed a set of eight few-shot exemplars with chains of thought for prompting—Figure 1 (right) shows one chain of thought exemplar, and the full set of exemplars is given in Appendix Table 20. (These particular exemplars did not undergo prompt engineering; robustness is studied in Section 3.4 and Appendix A.2.) To investigate whether chain-of-thought prompting in this form can successfully elicit successful reasoning across a range of

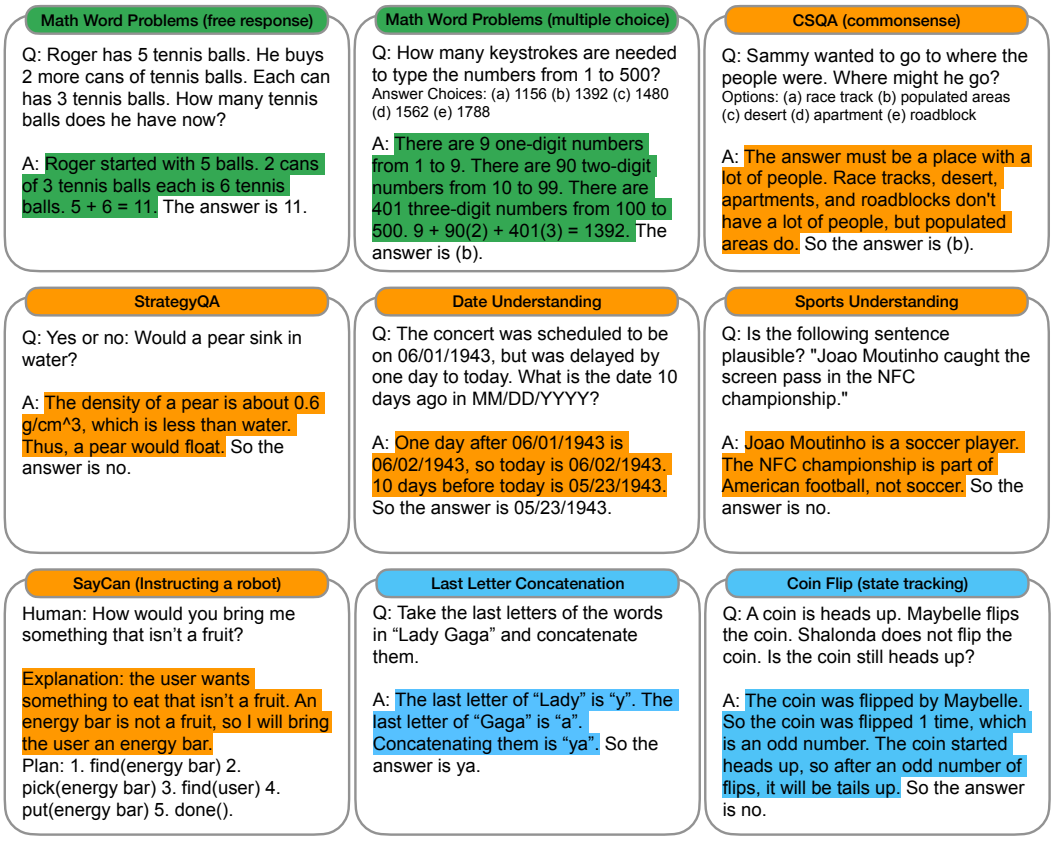


Figure 3: Examples of ⟨input, chain of thought, output⟩ triples for arithmetic, commonsense, and symbolic reasoning benchmarks. Chains of thought are highlighted. Full prompts in Appendix G.

math word problems, we used this single set of eight chain of thought exemplars for all benchmarks except AQuA, which is multiple choice instead of free response. For AQuA, we used four exemplars and solutions from the training set, as given in Appendix Table 21.

Language models. We evaluate five large language models. The first is **GPT-3** (Brown et al., 2020), for which we use text-ada-001, text-babbage-001, text-curie-001, and text-davinci-002, which presumably correspond to InstructGPT models of 350M, 1.3B, 6.7B, and 175B parameters (Ouyang et al., 2022).The second is **LaMDA** (Thoppilan et al., 2022), which has models of 422M, 2B, 8B, 68B, and 137B parameters. The third is **PaLM**, which has models of 8B, 62B, and 540B parameters. The fourth is **UL2 20B** (Tay et al., 2022), and the fifth is **Codex** (Chen et al., 2021, code-davinci-002 in the OpenAI API). We sample from the models via greedy decoding (though follow-up work shows chain-of-thought prompting can be improved by taking the majority final answer over many sampled generations (Wang et al., 2022a)). For LaMDA, we report averaged results over five random seeds, where each seed had a different randomly shuffled order of exemplars. As LaMDA experiments did not show large variance among different seeds, to save compute we report results for a single exemplar order for all other models.

3.2 Results

The strongest results of chain-of-thought prompting are summarized in Figure 4, with all experimental outputs for each model collection, model size, and benchmark shown in Table 2 in the Appendix. There are three key takeaways. First, Figure 4 shows that chain-of-thought prompting is an emergent ability of model scale (Wei et al., 2022b). That is, chain-of-thought prompting does not positively impact performance for small models, and only yields performance gains when used with models of $\sim$100B parameters. We qualitatively found that models of smaller scale produced fluent but illogical chains of thought, leading to lower performance than standard prompting.

Second, chain-of-thought prompting has larger performance gains for more-complicated problems. For instance, for GSM8K (the dataset with the lowest baseline performance), performance more than doubled for the largest GPT and PaLM models. On the other hand, for SingleOp, the easiest subset of MAWPS which only requires a single step to solve, performance improvements were either negative or very small (see Appendix Table 3).

Third, chain-of-thought prompting via GPT-3 175B and PaLM 540B compares favorably to prior state of the art, which typically finetunes a task-specific model on a labeled training dataset. Figure 4 shows how PaLM 540B uses chain-of-thought prompting to achieve new state of the art on GSM8K, SVAMP, and MAWPS (though note that standard prompting already passed the prior best for SVAMP). On the other two datasets, AQuA and ASDiv, PaLM with chain-of-thought prompting reaches within 2% of the state of the art (Appendix Table 2).

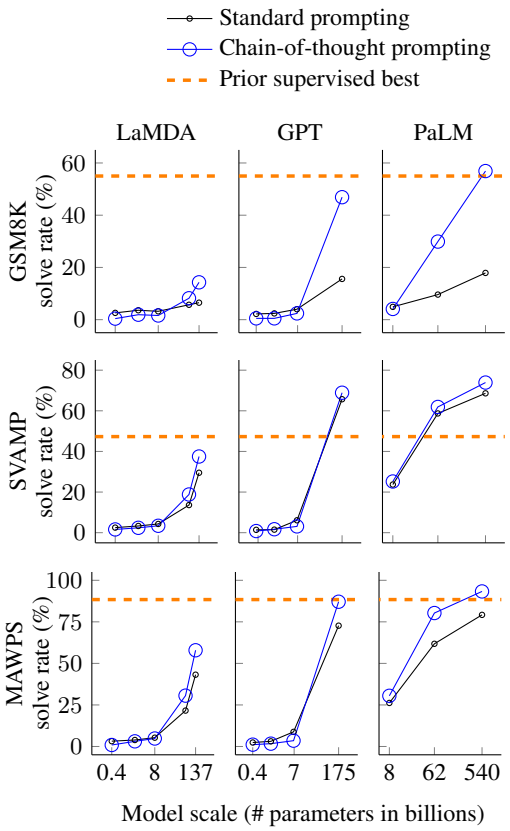


Figure 4: Chain-of-thought prompting enables large language models to solve challenging math problems. Notably, chain-of-thought reasoning is an emergent ability of increasing model scale. Prior best numbers are from Cobbe et al. (2021) for GSM8K, Jie et al. (2022) for SVAMP, and Lan et al. (2021) for MAWPS.

To better understand why chain-of-thought prompting works, we manually examined model-generated chains of thought by LaMDA 137B for GSM8K. Of 50 random examples where the model returned the correct final answer, all of the generated chains of thought were also logically and mathematically correct except two that coincidentally arrived at the correct answer (see Appendix D.1, and Table 8 for examples of correct model-generated chains of thought). We also randomly examined 50 random samples for which the model gave the wrong answer. The summary of this analysis is that 46% of the chains of thought were almost correct, barring minor mistakes (calculator error, symbol mapping error, or one reasoning step missing), and that the other 54% of the chains of thought had major errors in semantic understanding or coherence (see Appendix D.2). To provide a small insight into why scaling improves chain-of-thought reasoning ability, we performed a similar analysis of errors made by PaLM 62B and whether those errors were fixed by scaling to PaLM 540B. The summary is that scaling PaLM to 540B fixes a large portion of one-step missing and semantic understanding errors in the 62B model (see Appendix A.1).

3.3 Ablation Study

The observed benefits of using chain-of-thought prompting raises the natural question of whether the same performance improvements can be conferred via other types of prompting. Figure 5 shows an ablation study with three variations of chain of thought described below.

Equation only. One reason for why chain-of-thought prompting might help is that it produces the mathematical equation to be evaluated, and so we test a variation where the model is prompted to output only a mathematical equation before giving the answer. Figure 5 shows that equation only prompting does not help much for GSM8K, which implies that the semantics of the questions in GSM8K are too challenging to directly translate into an equation without the natural language reasoning steps in chain of thought. For datasets of one-step or two-step problems, however, we find that equation only prompting does improve performance, since the equation can be easily derived from the question (see Appendix Table 6).

Variable compute only. Another intuition is that chain of thought allows the model to spend more computation (i.e., intermediate tokens) on harder problems. To isolate the effect of variable computation from chain-of-thought reasoning, we test a configuration where the model is prompted to output a only sequence of dots (. . .) equal to the number of characters in the equation needed to solve the problem. This variant performs about the same as the baseline, which suggests that variable computation by itself is not the reason for the success of chain-of-thought prompting, and that there appears to be utility from expressing intermediate steps via natural language.

Chain of thought after answer. Another potential benefit of chain-of-thought prompting could simply be that such prompts allow the model to better access relevant knowledge acquired during pretraining. Therefore, we test an alternative configuration where the chain of thought prompt is only given after the answer, isolating whether the model actually depends on the produced chain of thought to give the final answer. This variant performs about the same as the baseline, which suggests that the sequential reasoning embodied in the chain of thought is useful for reasons beyond just activating knowledge.

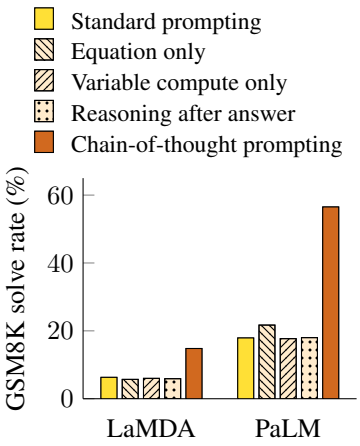


Figure 5: Ablation study for different variations of prompting using LaMDA 137B and PaLM 540B. Results for other datasets are given in Appendix Table 6 and Table 7.

3.4 Robustness of Chain of Thought

Sensitivity to exemplars is a key consideration of prompting approaches—for instance, varying the permutation of few-shot exemplars can cause the accuracy of GPT-3 on SST-2 to range from near chance (54.3%) to near state of the art (93.4%) (Zhao et al., 2021). In this final subsection, we evaluate robustness to chains of thought written by different annotators. In addition to the results above, which used chains of thought written by an Annotator A, two other co-authors of this paper (Annotators B and C) independently wrote chains of thought for the same few-shot exemplars (shown in Appendix H). Annotator A also wrote another chain of thought that was more concise than the original, following the style of solutions given in Cobbe et al. (2021).[1]

Figure 6 shows these results for LaMDA 137B on GSM8K and MAWPS (ablation results for other datasets are given in Appendix Table 6 / Table 7). Although there is variance among different chain of thought annotations, as would be expected when using exemplar-based prompting (Le Scao and Rush, 2021; Reynolds and McDonell, 2021; Zhao et al., 2021), all sets of chain of thought prompts outperform the standard baseline by a large margin. This result implies that successful use of chain of thought does not depend on a particular linguistic style.

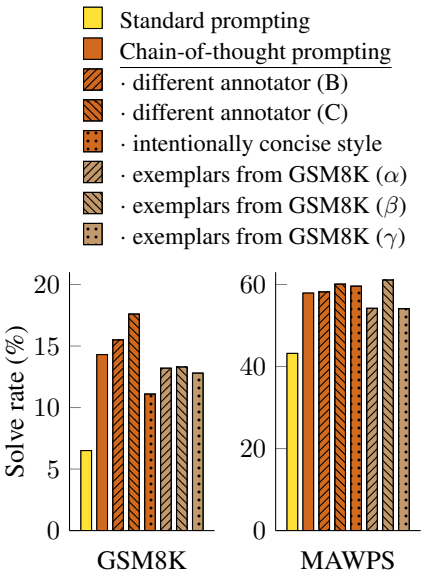


Figure 6: Chain-of-thought prompting has variance for different prompt examples (as expected) but outperforms standard prompting for various annotators as well as for different exemplars.

To confirm that successful chain-of-thought prompting works for other sets of exemplars, we also run experiments with three sets of eight exemplars randomly sampled from the GSM8K training set, an independent

[1]For instance, whereas original chain of thought uses several short sentences (*"'There were originally 9 computers. For each of 4 days, 5 more computers were added. So 5 * 4 = 20 computers were added. 9 + 20 is 29."*), the concise chain of thought would read *"5 * 4 = 20 new computers were added. So there are 9 + 20 = 29 new computers in the server room now"*.

source (examples in this dataset already included reasoning steps like a chain of thought).[2] Figure 6 shows that these prompts performed comparably with our manually written exemplars, also substantially outperforming standard prompting.

In addition to robustness to annotators, independently-written chains of thought, different exemplars, and various language models, we also find that chain-of-thought prompting for arithmetic reasoning is robust to different exemplar orders and varying numbers of exemplars (see Appendix A.2).

4 Commonsense Reasoning

Although chain of thought is particularly suitable for math word problems, the language-based nature of chain of thought actually makes it applicable to a broad class of commonsense reasoning problems, which involve reasoning about physical and human interactions under the presumption of general background knowledge. Commonsense reasoning is key for interacting with the world and is still beyond the reach of current natural language understanding systems (Talmor et al., 2021).

Benchmarks. We consider five datasets covering a diverse range of commonsense reasoning types. The popular **CSQA** (Talmor et al., 2019) asks commonsense questions about the world involving complex semantics that often require prior knowledge. **StrategyQA** (Geva et al., 2021) requires models to infer a multi-hop strategy to answer questions. We choose two specialized evaluation sets from the BIG-bench effort (BIG-bench collaboration, 2021): **Date** Understanding, which involves inferring a date from a given context, and **Sports** Understanding, which involves determining whether a sentence relating to sports is plausible or implausible. Finally, the **SayCan** dataset (Ahn et al., 2022) involves mapping a natural language instruction to a sequence of robot actions from a discrete set. Figure 3 shows examples with chain of thought annotations for all datasets.

Prompts. We follow the same experimental setup as the prior section. For CSQA and StrategyQA, we randomly selected examples from the training set and manually composed chains of thought for them to use as few-shot exemplars. The two BIG-bench tasks do not have training sets, so we selected the first ten examples as exemplars in the evaluation set as few-shot exemplars and report numbers on the rest of the evaluation set. For SayCan, we use six examples from the training set used in Ahn et al. (2022) and also manually composed chains of thought.

Results. Figure 7 highlights these results for PaLM (full results for LaMDA, GPT-3, and different model scales are shown in Table 4). For all tasks, scaling up model size improved the performance of standard prompting; chain-of-thought prompting led to further gains, with improvements appearing to be largest for PaLM 540B. With chain-of-thought prompting, PaLM 540B achieved strong performance relative to baselines, outperforming the prior state of the art on StrategyQA (75.6% vs 69.4%) and outperforming an unaided sports enthusiast on sports understanding (95.4% vs 84%). These results demonstrate that chain-of-thought prompting can also improve performance on tasks requiring a range of commonsense reasoning abilities (though note that gain was minimal on CSQA).

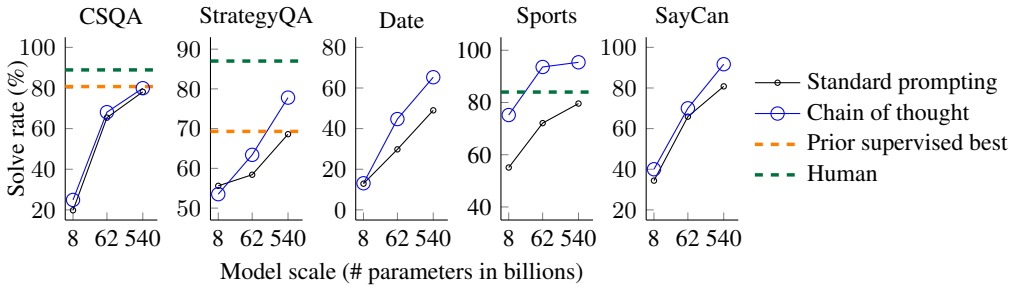


Figure 7: Chain-of-thought prompting also improves the commonsense reasoning abilities of language models. The language model shown here is PaLM. Prior best numbers are from the leaderboards of CSQA (Talmor et al., 2019) and StrategyQA (Geva et al., 2021) (single-model only, as of May 5, 2022). Additional results using various sizes of LaMDA, GPT-3, and PaLM are shown in Table 4.

[2]We sample examples ≤ 60 tokens to fit into our input context window, and also limit the examples to ≤ 2 steps to solve for a fair comparison with the eight exemplars that we composed.

5 Symbolic Reasoning

Our final experimental evaluation considers symbolic reasoning, which is simple for humans but potentially challenging for language models. We show that chain-of-thought prompting not only enables language models to perform symbolic reasoning tasks that are challenging in the standard prompting setting, but also facilitates length generalization to inference-time inputs longer than those seen in the few-shot exemplars.

Tasks. We use the following two toy tasks.

- **Last letter concatenation.** This task asks the model to concatenate the last letters of words in a name (e.g., *"Amy Brown"* → *"yn"*). It is a more challenging version of first letter concatenation, which language models can already perform without chain of thought.[3] We generate full names by randomly concatenating names from the top one-thousand first and last names from name census data (`https://namecensus.com/`).
- **Coin flip.** This task asks the model to answer whether a coin is still heads up after people either flip or don't flip the coin (e.g., *"A coin is heads up. Phoebe flips the coin. Osvaldo does not flip the coin. Is the coin still heads up?"* → *"no"*).

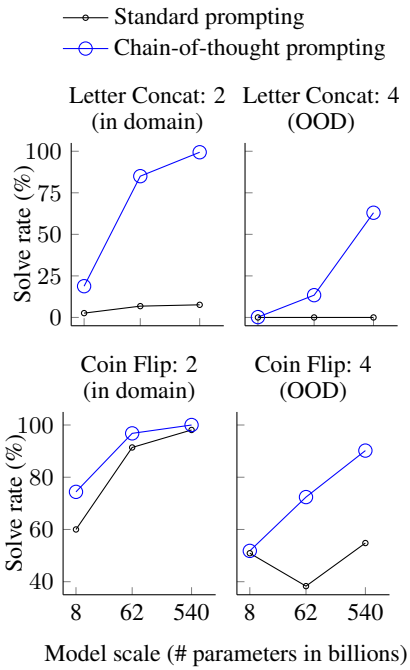


Figure 8: Using chain-of-thought prompting facilitates generalization to longer sequences in two symbolic reasoning tasks.

As the construction of these symbolic reasoning tasks is well-defined, for each task we consider an *in-domain* test set for which examples had the same number of steps as the training/few-shot exemplars, as well as an *out-of-domain* (OOD) test set, for which evaluation examples had more steps than those in the exemplars. For last letter concatenation, the model only sees exemplars of names with two words, and then performs last letter concatenation on names with 3 and 4 words.[4] We do the same for the number of potential flips in the coin flip task. Our experimental setup uses the same methods and models as in the prior two sections. We again manually compose chains of thought for the few-shot exemplars for each task, which are given in Figure 3.

Results. The results of these in-domain and OOD evaluations are shown in Figure 8 for PaLM, with results for LaMDA shown in Appendix Table 5. With PaLM 540B, chain-of-thought prompting leads to almost 100% solve rates (note that standard prompting already solves coin flip with PaLM 540, though not for LaMDA 137B). Note that these in-domain evaluations are "toy tasks" in the sense that perfect solution structures are already provided by the chains of thought in the few-shot exemplars; all the model has to do is repeat the same steps with the new symbols in the test-time example. And yet, small models still fail—the ability to perform abstract manipulations on unseen symbols for these three tasks only arises at the scale of 100B model parameters.

As for the OOD evaluations, standard prompting fails for both tasks. With chain-of-thought prompting, language models achieve upward scaling curves (though performance is lower than in the in-domain setting). Hence, chain-of-thought prompting facilitates length generalization beyond seen chains of thought for language models of sufficient scale.

6 Discussion

We have explored chain-of-thought prompting as a simple mechanism for eliciting multi-step reasoning behavior in large language models. We first saw that chain-of-thought prompting improves performance by a large margin on arithmetic reasoning, yielding improvements that are much stronger than ablations and robust to different annotators, exemplars, and language models (Section 3). Next,

[3] We tested 10 common names using GPT-3 `davinci` and it got all but one correct.

[4] For names of length longer than 2 words, we concatenate multiple first and last names together.

experiments on commonsense reasoning underscored how the linguistic nature of chain-of-thought reasoning makes it generally applicable (Section 4). Finally, we showed that for symbolic reasoning, chain-of-thought prompting facilitates OOD generalization to longer sequence lengths (Section 5). In all experiments, chain-of-thought reasoning is elicited simply by prompting an off-the-shelf language model. No language models were finetuned in the process of writing this paper.

The emergence of chain-of-thought reasoning as a result of model scale has been a prevailing theme (Wei et al., 2022b). For many reasoning tasks where standard prompting has a flat scaling curve, chain-of-thought prompting leads to dramatically increasing scaling curves. Chain-of-thought prompting appears to expand the set of tasks that large language models can perform successfully—in other words, our work underscores that standard prompting only provides a lower bound on the capabilities of large language models. This observation likely raises more questions than it answers—for instance, how much more can we expect reasoning ability to improve with a further increase in model scale? What other prompting methods might expand the range of tasks that language models can solve?

As for limitations, we first qualify that although chain of thought emulates the thought processes of human reasoners, this does not answer whether the neural network is actually "reasoning," which we leave as an open question. Second, although the cost of manually augmenting exemplars with chains of thought is minimal in the few-shot setting, such annotation costs could be prohibitive for finetuning (though this could potentially be surmounted with synthetic data generation, or zero-shot generalization). Third, there is no guarantee of correct reasoning paths, which can lead to both correct and incorrect answers; improving factual generations of language models is an open direction for future work (Rashkin et al., 2021; Ye and Durrett, 2022; Wiegreffe et al., 2022, *inter alia*). Finally, the emergence of chain-of-thought reasoning only at large model scales makes it costly to serve in real-world applications; further research could explore how to induce reasoning in smaller models.

7 Related Work

This work is inspired by many research areas, which we detail in an extended related work section (Appendix C). Here we describe two directions and associated papers that are perhaps most relevant.

The first relevant direction is using intermediate steps to solve reasoning problems. Ling et al. (2017) pioneer the idea of using natural language rationales to solve math word problems through a series of intermediate steps. Their work is a remarkable contrast to the literature using formal languages to reason (Roy et al., 2015; Chiang and Chen, 2019; Amini et al., 2019; Chen et al., 2019). Cobbe et al. (2021) extend Ling et al. (2017) by creating a larger dataset and using it to finetune a pretrained language model rather than training a model from scratch. In the domain of program synthesis, Nye et al. (2021) leverage language models to predict the final outputs of Python programs via first line-to-line predicting the intermediate computational results, and show that their step-by-step prediction method performs better than directly predicting the final outputs.

Naturally, this paper also relates closely to the large body of recent work on prompting. Since the popularization of few-shot prompting as given by Brown et al. (2020), several general approaches have improved the prompting ability of models, such as automatically learning prompts (Lester et al., 2021) or giving models instructions describing a task (Wei et al., 2022a; Sanh et al., 2022; Ouyang et al., 2022). Whereas these approaches improve or augment the input part of the prompt (e.g., instructions that are prepended to inputs), our work takes the orthogonal direction of augmenting the outputs of language models with a chain of thought.

8 Conclusions

We have explored chain-of-thought prompting as a simple and broadly applicable method for enhancing reasoning in language models. Through experiments on arithmetic, symbolic, and commonsense reasoning, we find that chain-of-thought reasoning is an emergent property of model scale that allows sufficiently large language models to perform reasoning tasks that otherwise have flat scaling curves. Broadening the range of reasoning tasks that language models can perform will hopefully inspire further work on language-based approaches to reasoning.